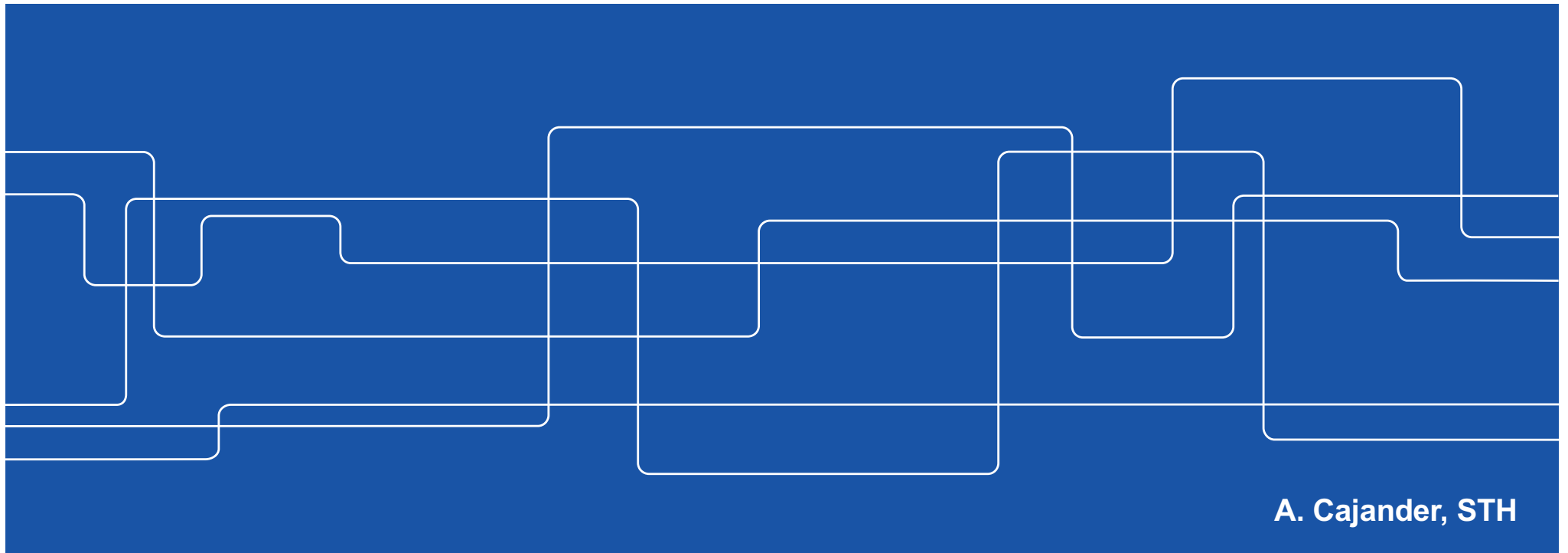




HE1025 Operativsystem

F4: Virtualisering Minnet - Paging



A. Cajander, STH



Kursen...

F1: V#1 Processorn.

Ö1: V#2 c Pekare...

F2: V#3 Schemaläggning

L1: V#4 FreeRTOS

F3: V#5 Segmentation

Ö2: Buffer

F4: V#7 Paging

L2: V#8 Paging

F5: C#1 Locks & CV

Ö3: C#2 LINUX

F6: C#3 Semaphores

L3: C#4 Barberare

F7: P#1 Storage

Ö4: P#2 Buffer

F8: P#3 Communication

L4: P#4 Client/Server

Kap (1) 2..4 & 6

Kap 14 & c-boken

Kap 7..9,(10-11), RTOS

Lab #1 Handledning

Kap 12..13, 15..17

Kap 18-22, (23-24)

Lab #2 Handledning

Kap (25) 26..27 & 28..29.1

Kap 5 (39)

Kap 30..33, (34)

Lab #3 Handledning

Kap (35,) 36, 40, 42 (37..39, 41 & 43..46)

t.b.d

Kap (47, 39) & 48, (49..51)

Lab #4 Handledning

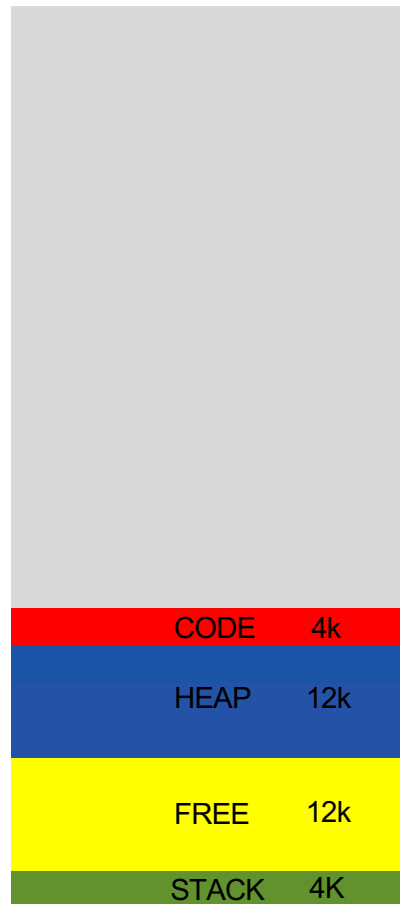
TEN1: 4+4p/4:e-del, minst 4p per 4:e-del för E!

LAB1: Närvaro L1..4



Virtualisering av minnet, worst case...

96kB



32kB Process #1

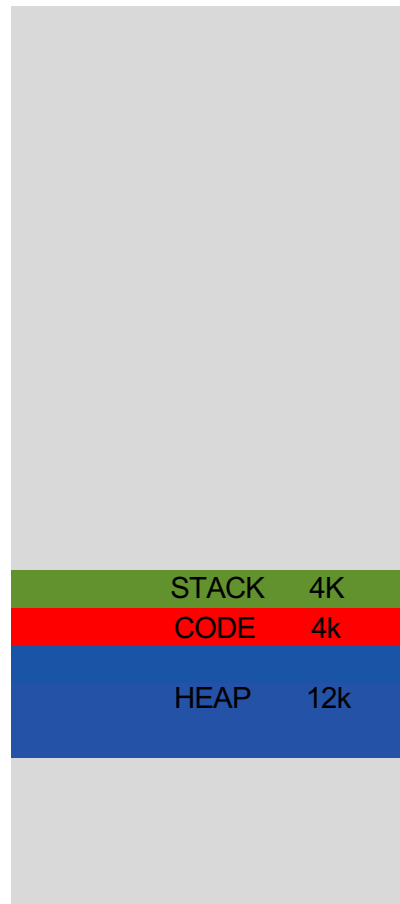
Innan en process blir running...
...måste hela dess adressrymd...
...kopieras in i det fysiska minnet!

I exemplet kan enbart 3 processer
exekvera till synes samtidigt utan
att processers hela minnesrymd
måste kopieras in/ut till disk!



Virtualisering av minnet, segmentering...

96kB



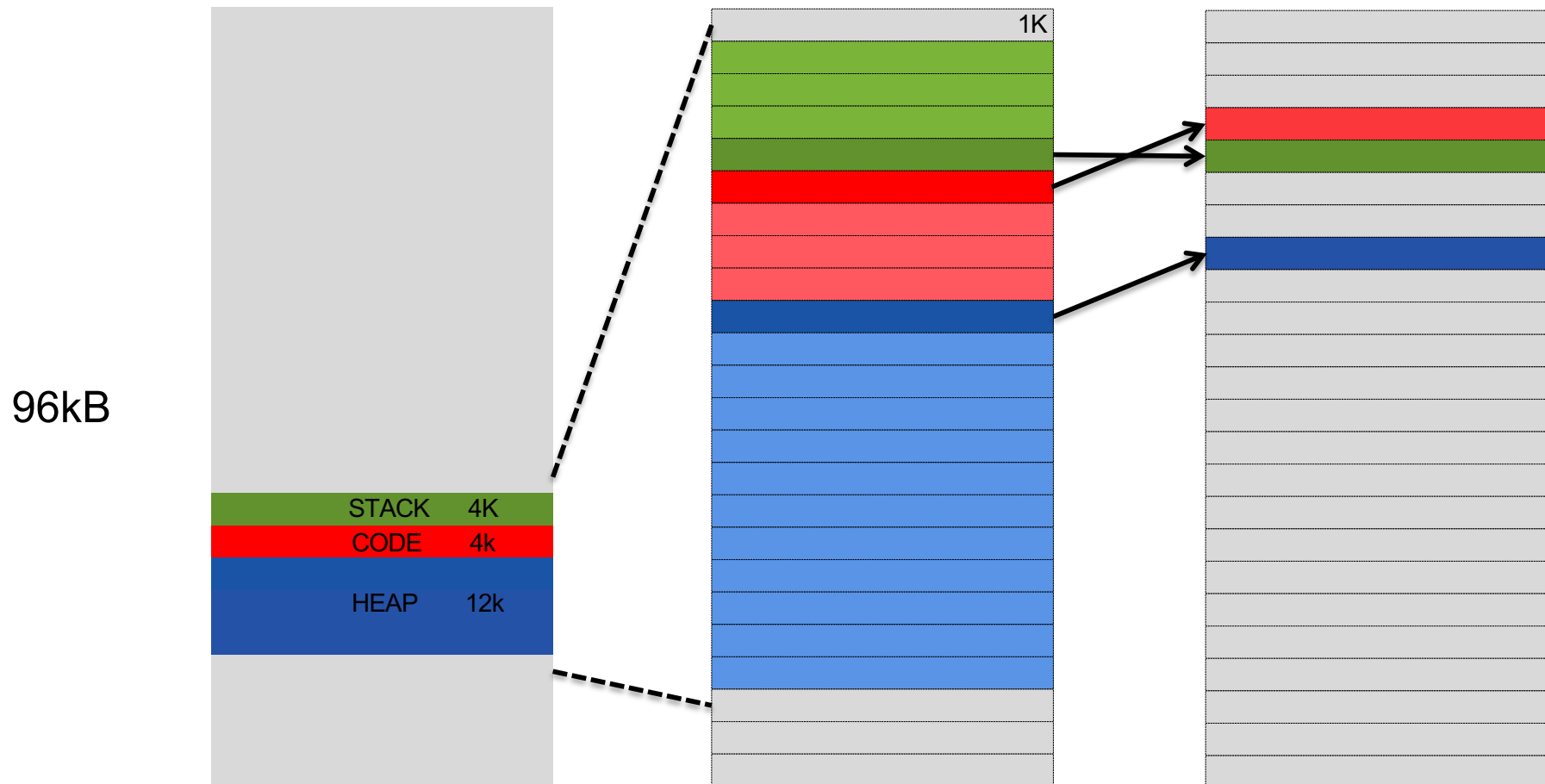
20kB Process #1

Med hjälp av segmenteringstekniken...
...går det att minska minnesbehovet...
...till segmentens faktiska storlek...
...som dock är variabel...
...och dessutom kan variera över tid!

Exempelprocessen tar i detta fall upp
12kB mindre fysiskt minne trots att den
upplevda adressrymden är densamma.

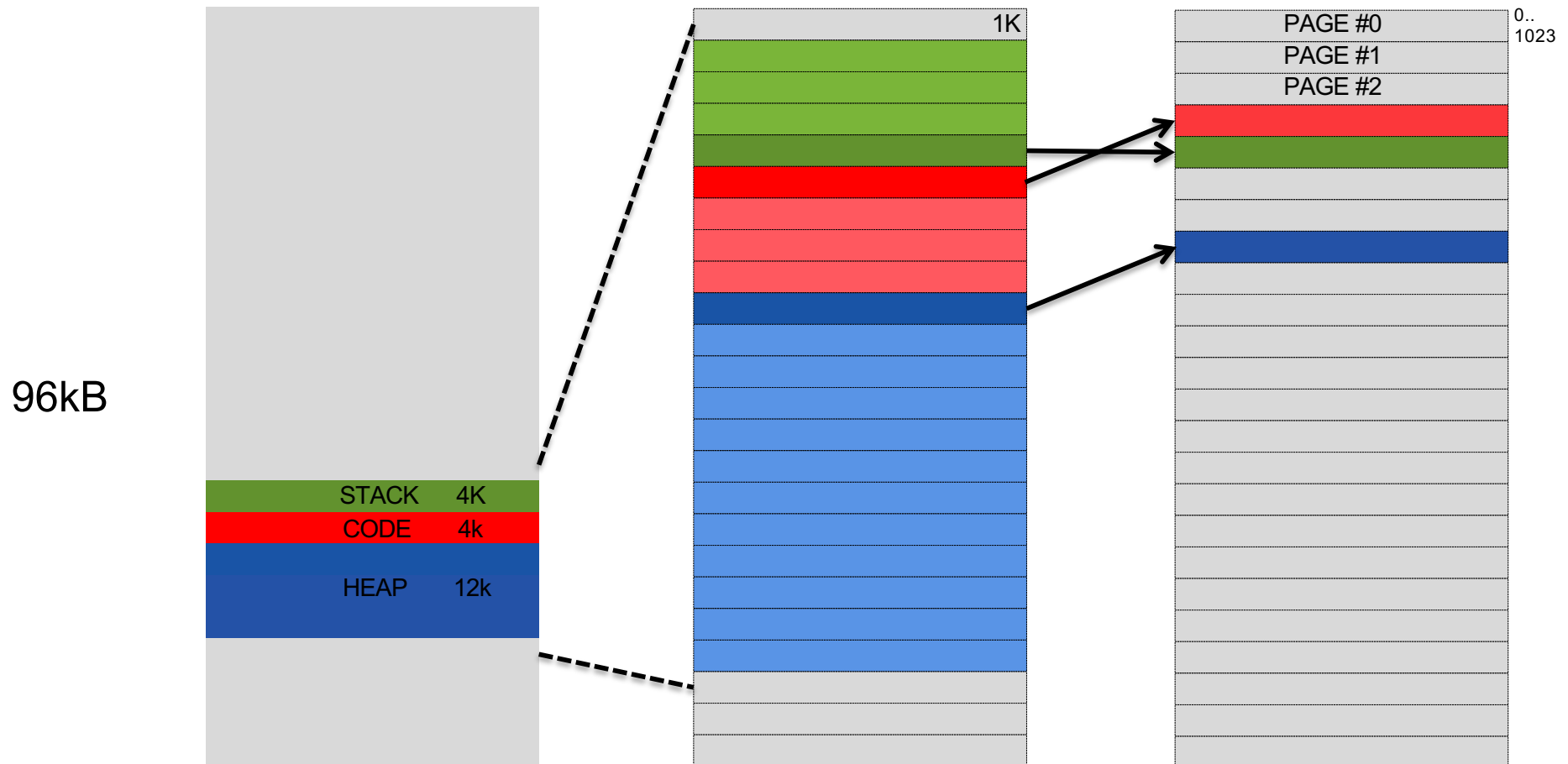
Det fria minnet blir dock lätt fragmenterat, finns det något annat sätt?

Virtualisering av minnet, sidor (paging)...



Tänk om det tillgängliga minnet ses som en mängd lika stora sidor...
...det fria minnet skulle kunna hanteras otroligt enkelt och bara "aktiva"
sidor av "running" segment måste finnas i det fysiska minnet!

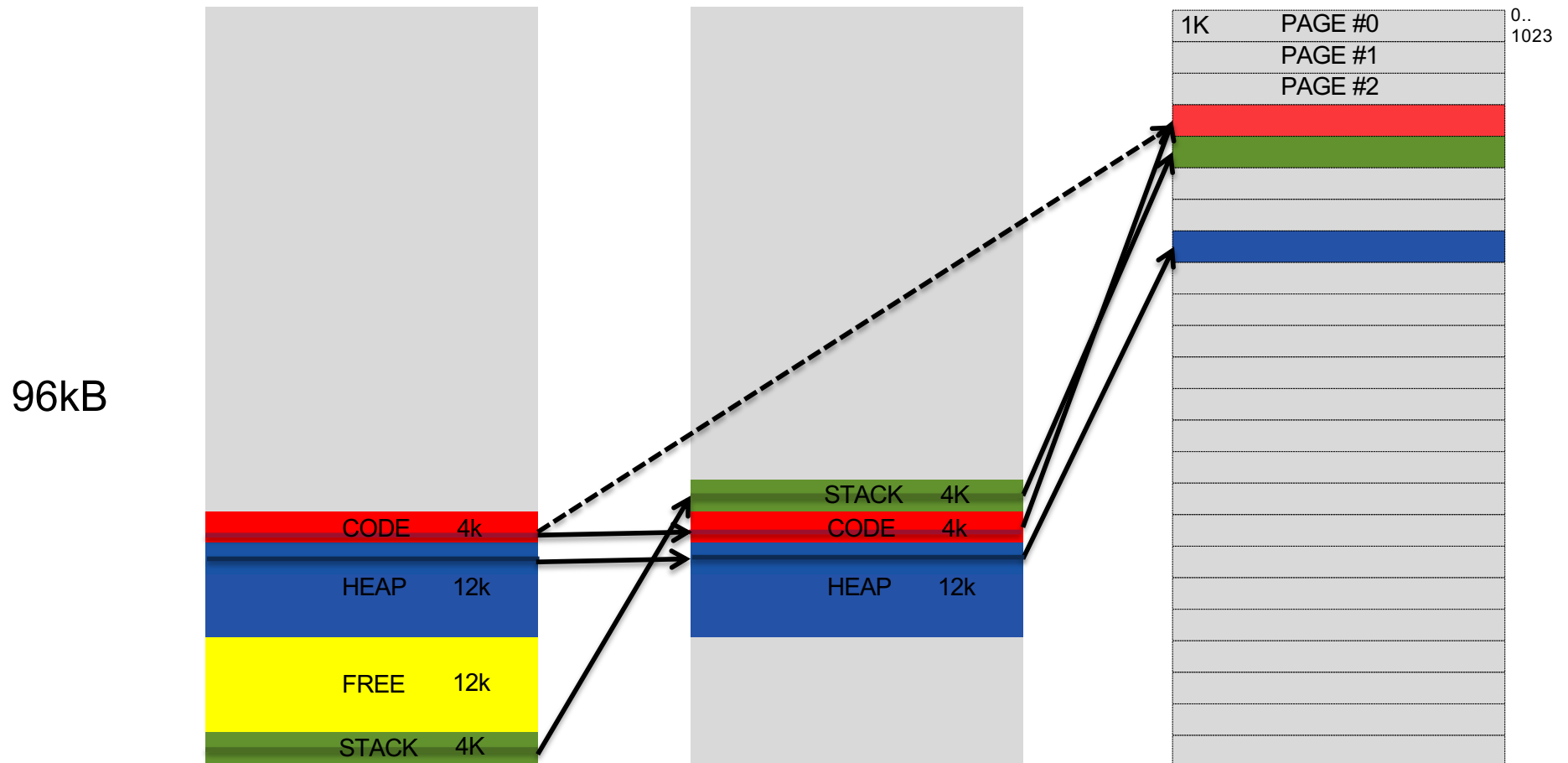
Virtualisering av minnet, sidor (paging)...



Adressen mest signifikanta del blir då ett sidnummer (page address) & den minst signifikanta delen en offset i sidan. I exemplet ovan blir offseten de 10 minst signifikanta bitarna (0..1023) och resten sidnummer!

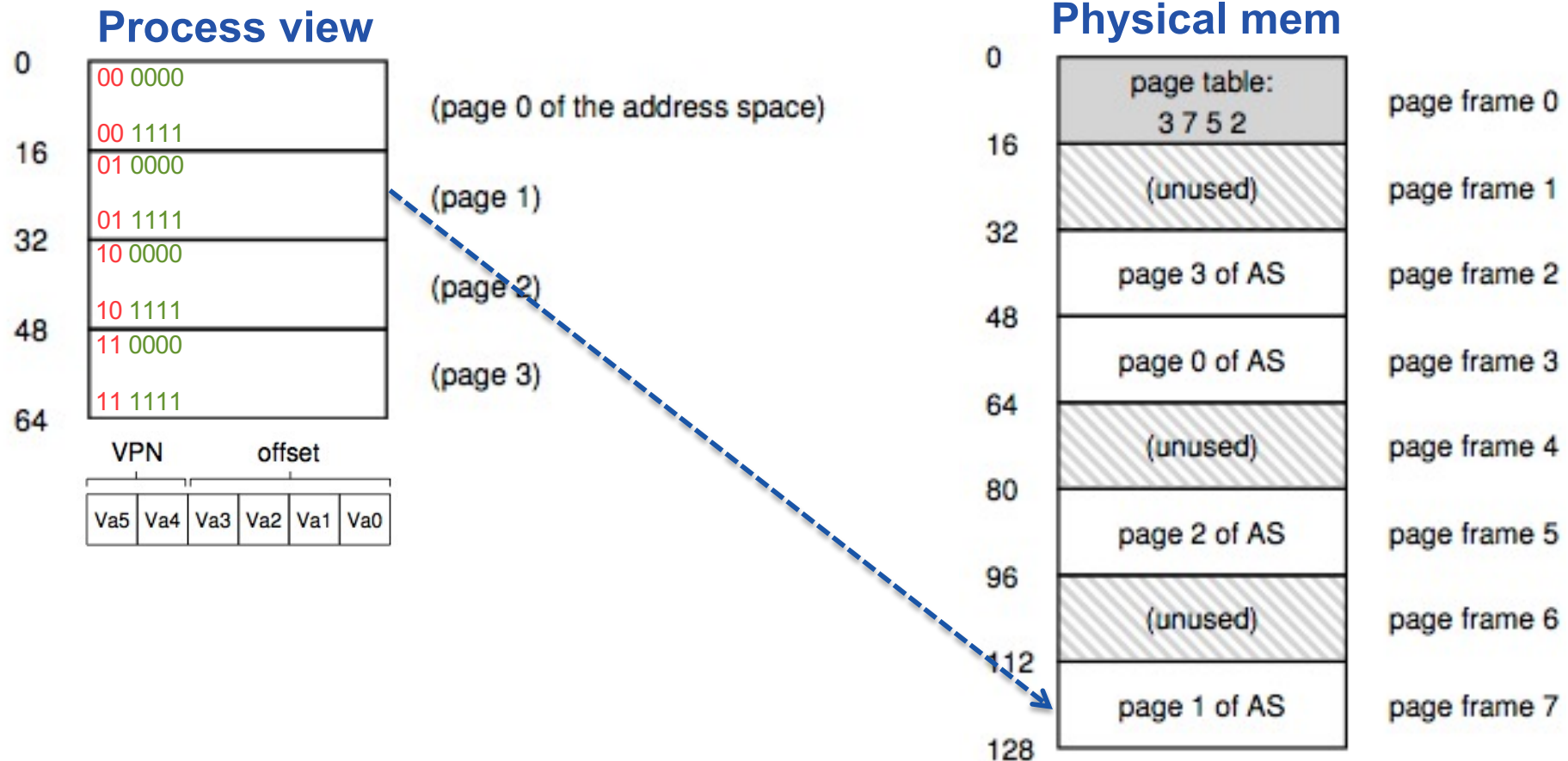


Virtualisering av minnet, sidor (paging)...



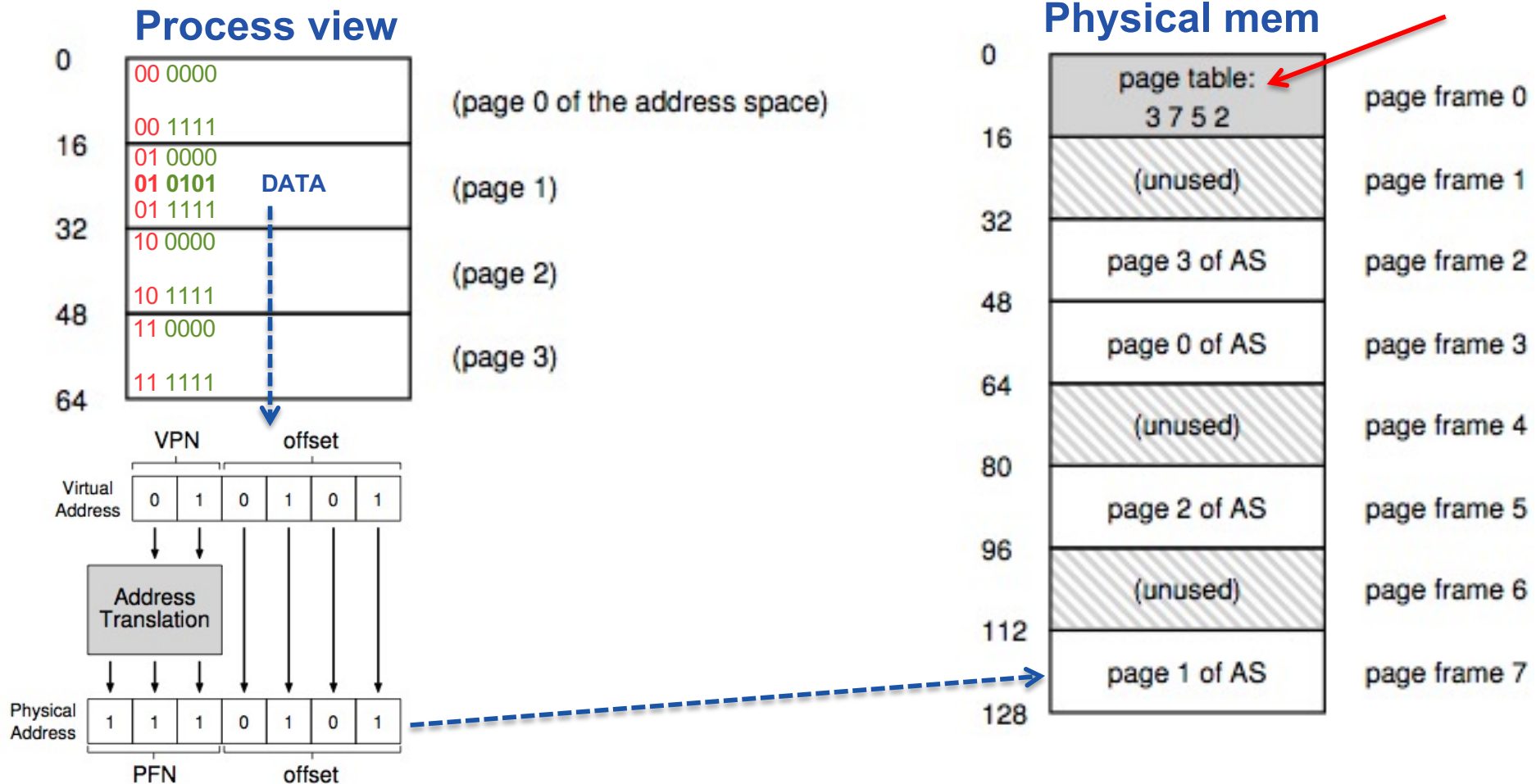
Sidtekniken kan användas tillsammans med segmenttekniken eller inte. Med sidtekniken spelar "hållet" i adressrymden ingen roll då enbart sidor som används tar plats i det fysiska minnet!

Virtual Page Nb, PPN & Page table...



Aha! Process adressrymd 64 Byte = 6 adressbitar, tänkt som 4 sidor...
...fysiskt minne med plats för 8 sidor (page frame)

Virtual Page Nb, PP/FN & Page table...

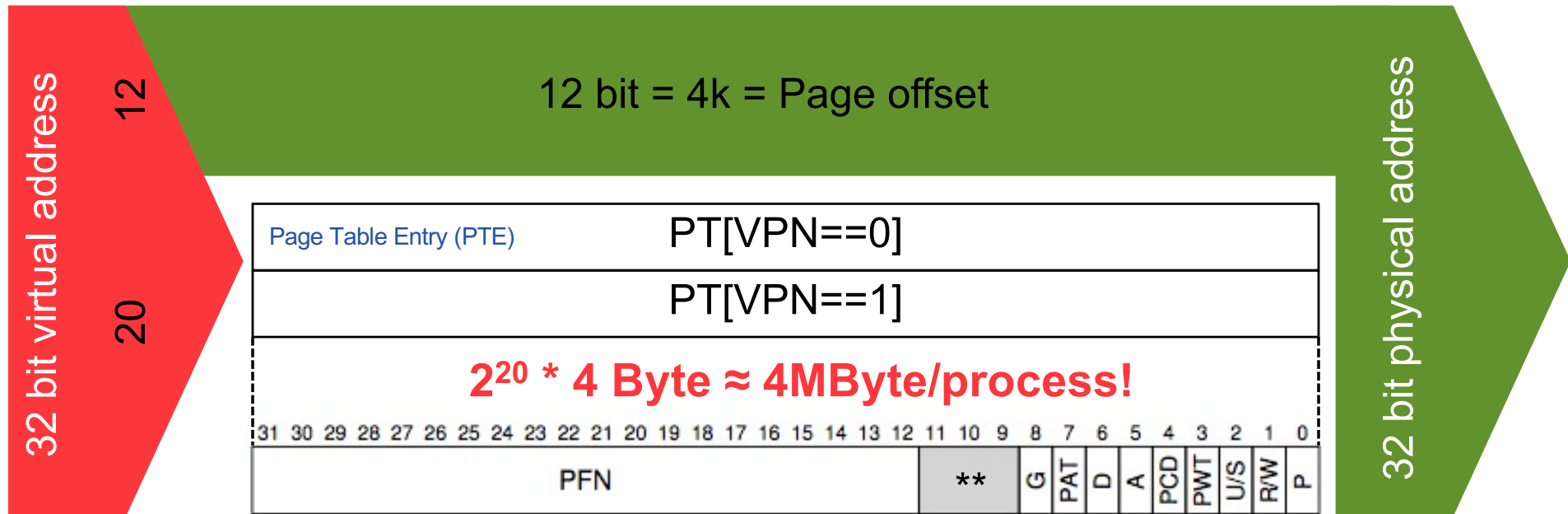


Virtual Page Number
Physical Frame Number

Virtuella adress översätts med hjälp av en sidtabell (**page table**), till korrekt fysiskt adress (page frame + samma offset)!
Varje process har en egen sidtabell...

(Linear) Page table?!

(Storleksuppskattning ingår i kursen)

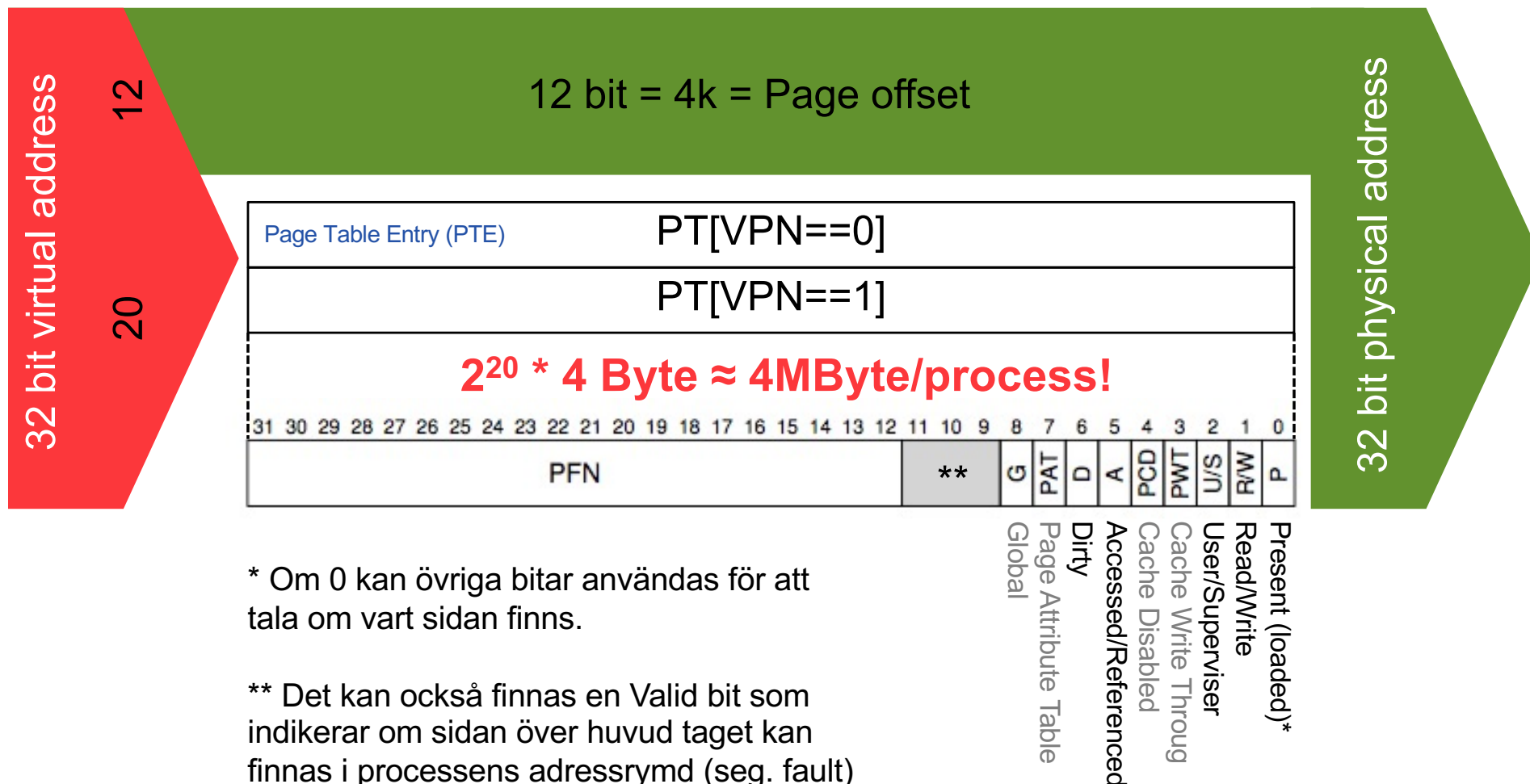


X86_64 arkitekturen: 48bit(256TiB) VA / 52bit(4PiB) PA

Linux: `cat /proc/cpuinfo`, Apple: `sysctl -a | grep machdep.cpu`

För stor för att hållas i dedicerat minne i processorn...
...speciellt PageTableBaseReg håller adressen till PT[0] för aktiv process.

(Linear) Page table?!



Det kan bli page fault, segmentation fault, protection fault samt att OS:et måste hålla reda på om sidan är skriven till och om den är "efterfrågad"!



För varje minnesaccess...

movl 21, %eax

```
1  // Extract the VPN from the virtual address
2  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
3
4  // Form the address of the page-table entry (PTE)
5  PTEAddr = PTBR + (VPN * sizeof(PTE))
6
7  // Fetch the PTE
8  PTE = AccessMemory(PTEAddr) ←
9
10 // Check if process can access the page
11 if (PTE.Valid == False)
12     RaiseException(SEGMENTATION_FAULT)
13 else if (CanAccess(PTE.ProtectBits) == False)
14     RaiseException(PROTECTION_FAULT)
15 else
16     // Access is OK: form physical address and fetch it
17     offset = VirtualAddress & OFFSET_MASK
18     PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
19     Register = AccessMemory(PhysAddr) ←
    +Manage usage bits in PTE...
```

Det blir en extra minnessaccess för varje minnesreferens...
Vi måste hantera **1) Beräkningsbördan & 2) Tabellens storlek!**



1) Beräkningsbördan

lakttagelser

```
int array[1000];  
...  
for (i = 0; i < 1000; i++)  
    array[i] = 0;
```

```
0 -> array[ i ]  
1024 movl $0x0, (%edi,%eax,4)  
1028 incl %eax          i++  
1032 cmpl $0x03e8,%eax  
1036 jne 0x1024          =1000?
```



lakttagelser

```
int array[1000];  
...  
for (i = 0; i < 1000; i++)  
    array[i] = 0;
```

0 -> array[i]

```
1024 movl $0x0, (%edi,%eax,4)  
1028 incl %eax i++  
1032 cmpl $0x03e8,%eax  
1036 jne 0x1024 =1000?
```

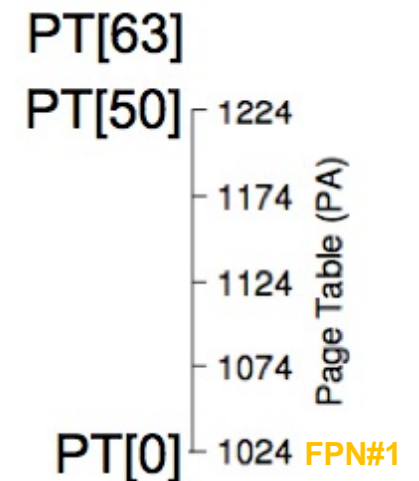
Virtual Address Space 64KB

Page size 1KB (VPN 0..63)

VVVV VVOO OOOO OOOO

Page Table [0] at PA 1024

lookup for VA 0..1023





lakttagelser

```
int array[1000];  
...  
for (i = 0; i < 1000; i++)  
    array[i] = 0;
```

0 -> array[i]

```
1024 movl $0x0, (%edi, %eax, 4)  
1028 incl %eax  
1032 cmpl $0x03e8, %eax  
1036 jne 0x1024
```

i++
=1000?

Virtual Address Space 64KB

Page size 1KB (VPN 0..63)

VVVV VVOO OOOO OOOO

Page Table [0] at PA 1024

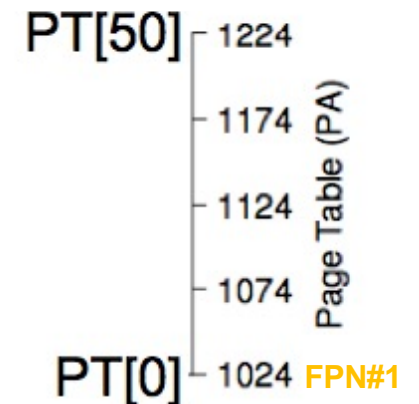
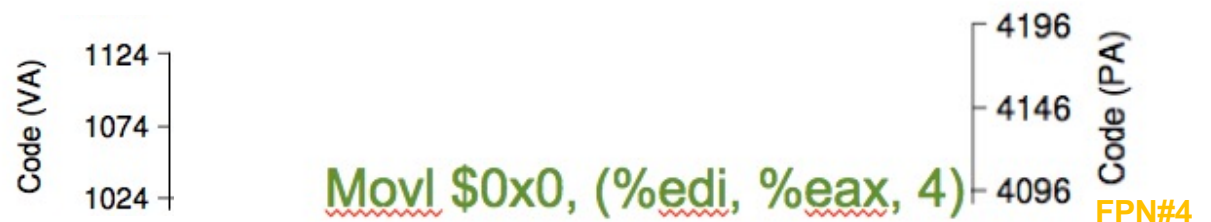
lookup for VA 0..1023

CS start at VA 1024...

thus VPN=1...

mapped to PFN=4...

thus PA 4096...5119





lakttagelser

```
int array[1000];  
...  
for (i = 0; i < 1000; i++)  
    array[i] = 0;
```

0 -> array[i]

```
1024 movl $0x0, (%edi, %eax, 4)  
1028 incl %eax  
1032 cmpl $0x03e8, %eax  
1036 jne 0x1024
```

i++
=1000?

Virtual Address Space 64KB

Page size 1KB (VPN 0..63)

VVVV VVOO OOOO OOOO

Page Table [0] at PA 1024

lookup for VA 0..1023

CS start at VA 1024...

thus VPN=1...

mapped to PFN=4...

thus PA 4096...5119

SS starts at VA 39936...

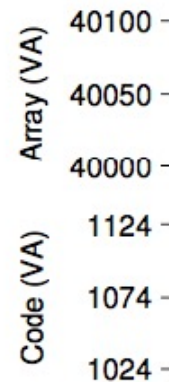
thus VPN=39...

mapped to PFN=7...

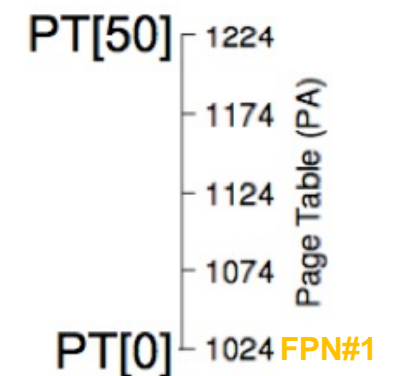
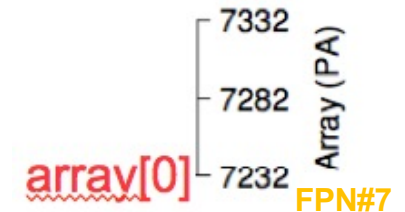
thus PA 7168...

array at VA 40000...

thus PA 7232!



Movl \$0x0, (%edi, %eax, 4)





lakttagelser

```
int array[1000];  
...  
for (i = 0; i < 1000; i++)  
    array[i] = 0;
```

0 -> array[i]
1024 movl \$0x0, (%edi,%eax,4)
1028 incl %eax
1032 cmpl \$0x03e8,%eax
1036 jne 0x1024
i++
=1000?

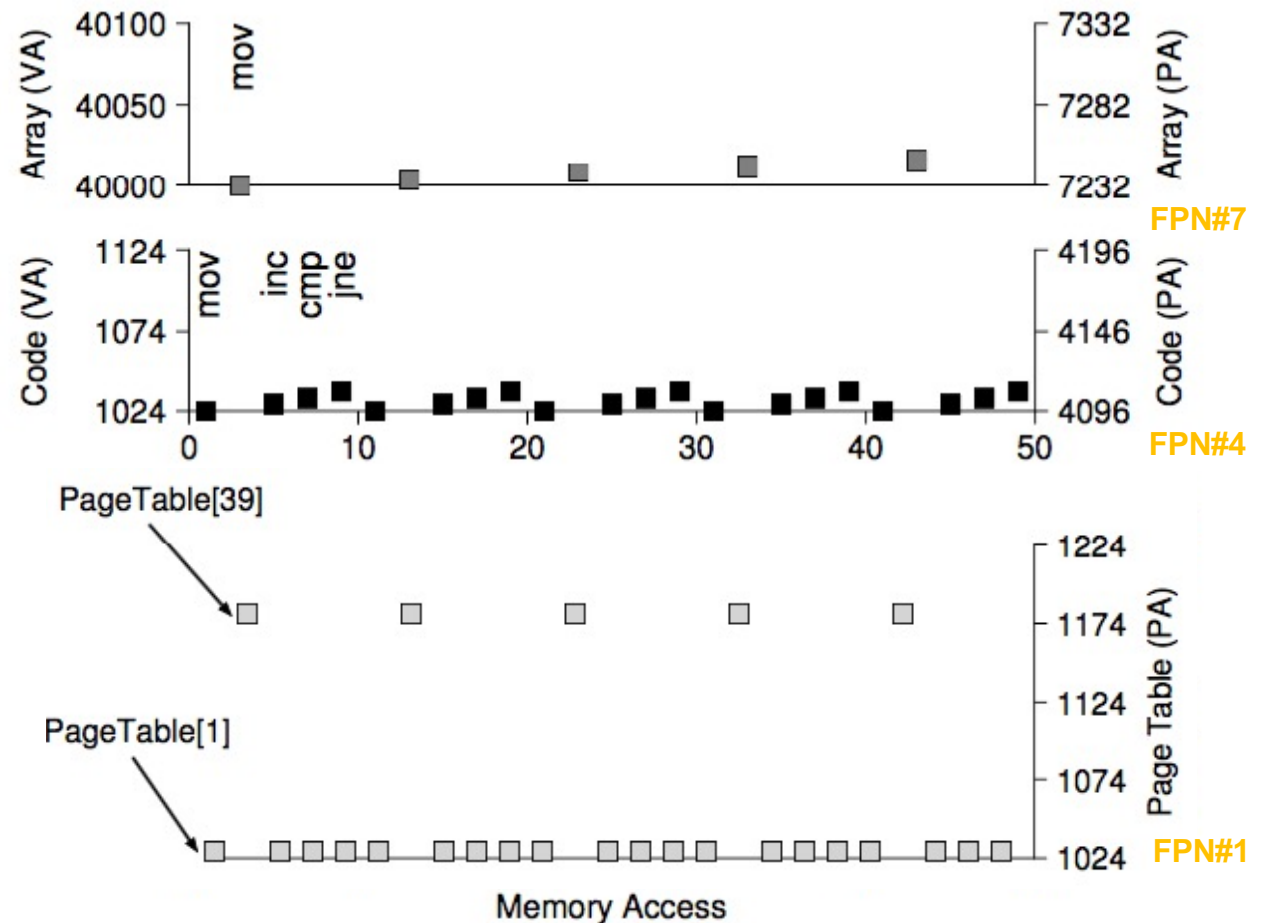
Virtual Address Space 64KB

Page size 1KB (VPN 0..63)

Page Table [0] at PA 1024
lookup for VA 0..1023

CS start at VA 1024...
thus VPN=1...
mapped to PFN=4...
thus PA 4096...5119

SS starts at VA 39936...
thus VPN=39...
mapped to PFN=7...
thus PA 7168...
array at VA 40000...
thus PA 7232!



Under hela "loopen" används enbart 2 PTE:s...
...programkod har ofta vad som benämns för en lokalitet!



1) Beräkningsbördan

Translation look-aside buffer (TLB)!

aka address-translation cache

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN) 4096B Content-addressable mem.
3  if (Success == True)    // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else    // TLB Miss
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)
16         RaiseException(PROTECTION_FAULT)
17     else
18         TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19     RetryInstruction()
```

Associativt minne, VPN IN -> PFN UT om "träff"



1) Beräkningsbördan

Translation look-aside buffer (TLB)!

aka address-translation cache

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)    // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else // TLB Miss
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)
16         RaiseException(PROTECTION_FAULT)
17     else
18         TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19     RetryInstruction()
```

4096B Content-addressable mem.

+0.5-1

1 mem ref

"Hit" no penalty...



1) Beräkningsbördan

Translation look-aside buffer (TLB)!

aka address-translation cache

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)    // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else // TLB Miss
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)
16         RaiseException(PROTECTION_FAULT)
17     else
18         TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19     RetryInstruction()
```

4096B Content-addressable mem.

+0.5-1

1 mem ref

+10-100

ARM SW, X86 REG (CR3)

SW: Return to same instruction, infinit chain, flexible
HW: Inflexible, safe.



1) Beräkningsbördan

Translation look-aside buffer (TLB)!

aka address-translation cache

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)    // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else // TLB Miss
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)
16         RaiseException(PROTECTION_FAULT)
17     else
18         TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19     RetryInstruction()
```

4096B Content-addressable mem.

+0.5-1

1 mem ref

+10-100

ARM SW, X86 REG (CR3)

Spatial & temporal locality -> "hit rate" avr 99.0-99.99%



TLB Contents, på djupet...

Innehållet är främst VPN & PFN men även flaggbitar...

...främst flaggor från PTE men speciellt en extra bit:

Ofta har ett PTE en V(alid) bit som indikerar om sidan tillhör ett segment som används av processen...

...TLB har därutöver en V(alid) bit som helt enkel talar om övriga bitar innehåller en korrekt mappning eller ej...

...när OS:et startar, eller en process växlas in, så sätts alla V(alid)-bitar i TLB till 0 för att markera att samtliga poster kan bytas ut successivt mot nya mappningar!

De kvarvarande, icke tillämpliga, mappningarna innehåller trots allt information som i vissa fall kunnat utnyttjas av skadlig kod för dumheter.

TLB Contents, på djupet...

1) Beräkningsbördan

VPN	PFN	valid	prot	ASID
10	100	1	rwX	1
—	—	0	—	—
10	170	1	rwX	2
—	—	0	—	—

VPN	PFN	valid	prot	ASID
10	101	1	r-x	1
—	—	0	—	—
50	101	1	r-x	2
—	—	0	—	—

Del av TLB kan också vara reserverad för OS:et så att dess mest använda mappningar (förutom vid första accessen) alltid ger "HIT"!



1) Beräkningsbördan

TLB: Replacement Policy

Least-recently-used

- +Rimligt antagande
- Hålla koll på mätetalet
- Specialfall leder alltid till cash-miss

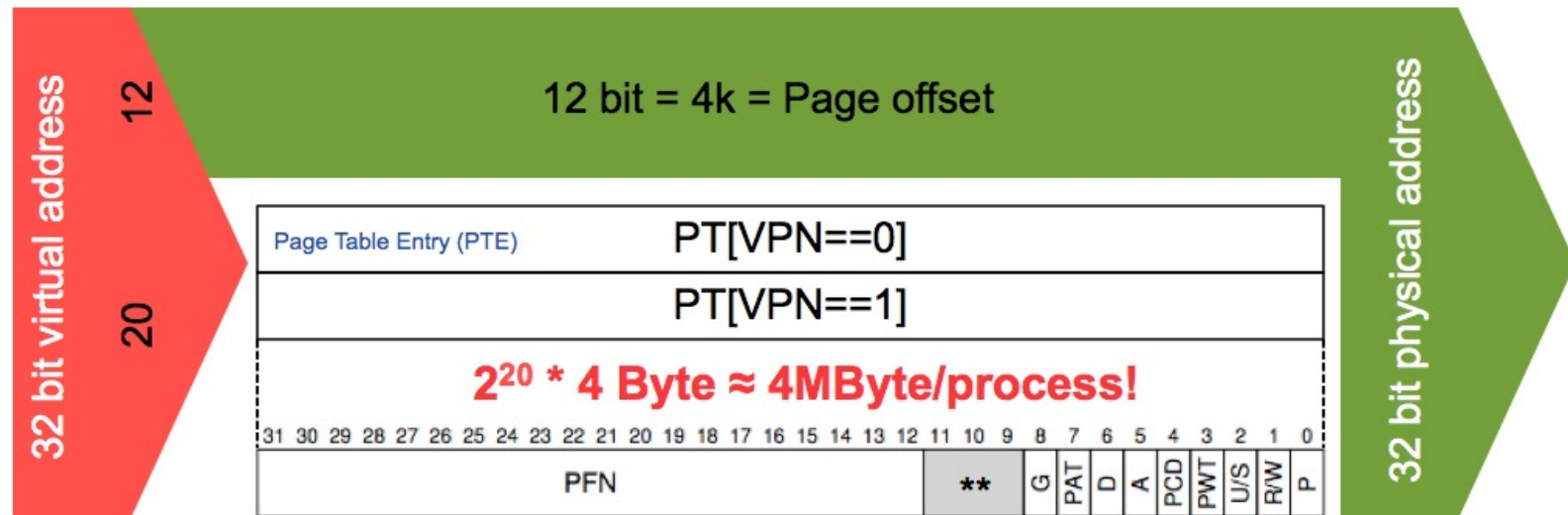
Random

- +Enkel att implementera
- +Behandlar alla användningsfall lika

OBS: Det finns en TLB per processor om det finns flera!

Större sidor -> Färre PTE -> mindre PT?!

2) Tabellstorleken



16k Sidor -> PT 1MByte/process

4M Sidor -> PT 8KByte/process

- + PT blir mindre, men framför allt
- + Det blir färre TLB missar (DB)
- Leder till intern fragmentering, "färre" tillgängliga PFN.

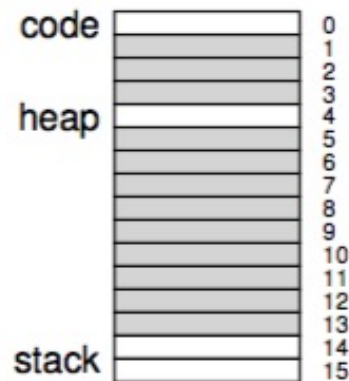
Större sidor är nog inte den ultimata lösningen...

...vi måste nog titta på hur vi lagrar själva sidtabellen?!

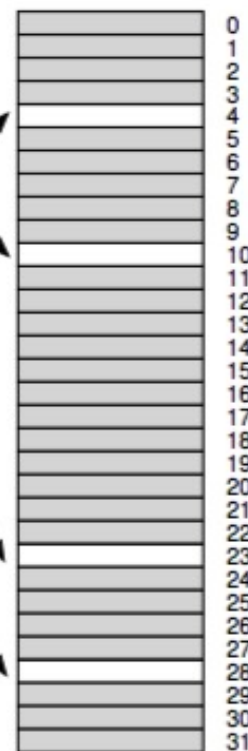
(Linear) Page table

2) Tabellstorleken

Virtual Address Space



Physical Memory



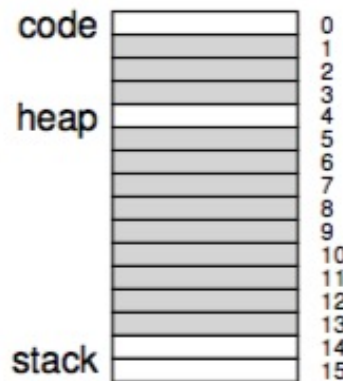
PFN	valid	prot	present	dirty
10	1	r-x	1	0
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
23	1	rw-	1	1
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
28	1	rw-	1	1
4	1	rw-	1	1

Hum, nästan alla poster i PT är invalid?!

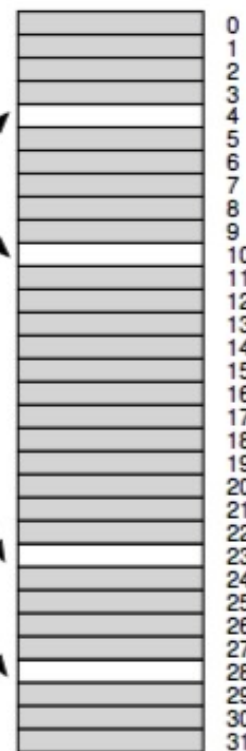
Page Table, trick #1: Hybridlösning!

2) Tabellstorleken

Virtual Address Space



Physical Memory

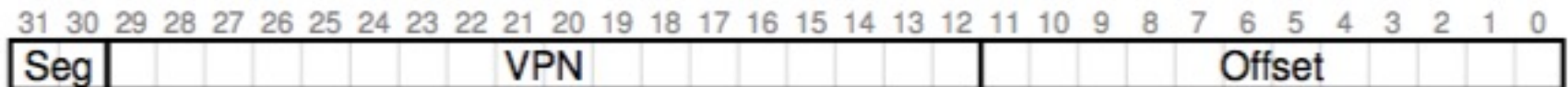


Varje process
får 4 små PT!

PFN	valid	prot	present	dirty
10	1	r-x	1	0
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
23	1	rw-	1	1
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
28	1	rw-	1	1
4	1	rw-	1	1

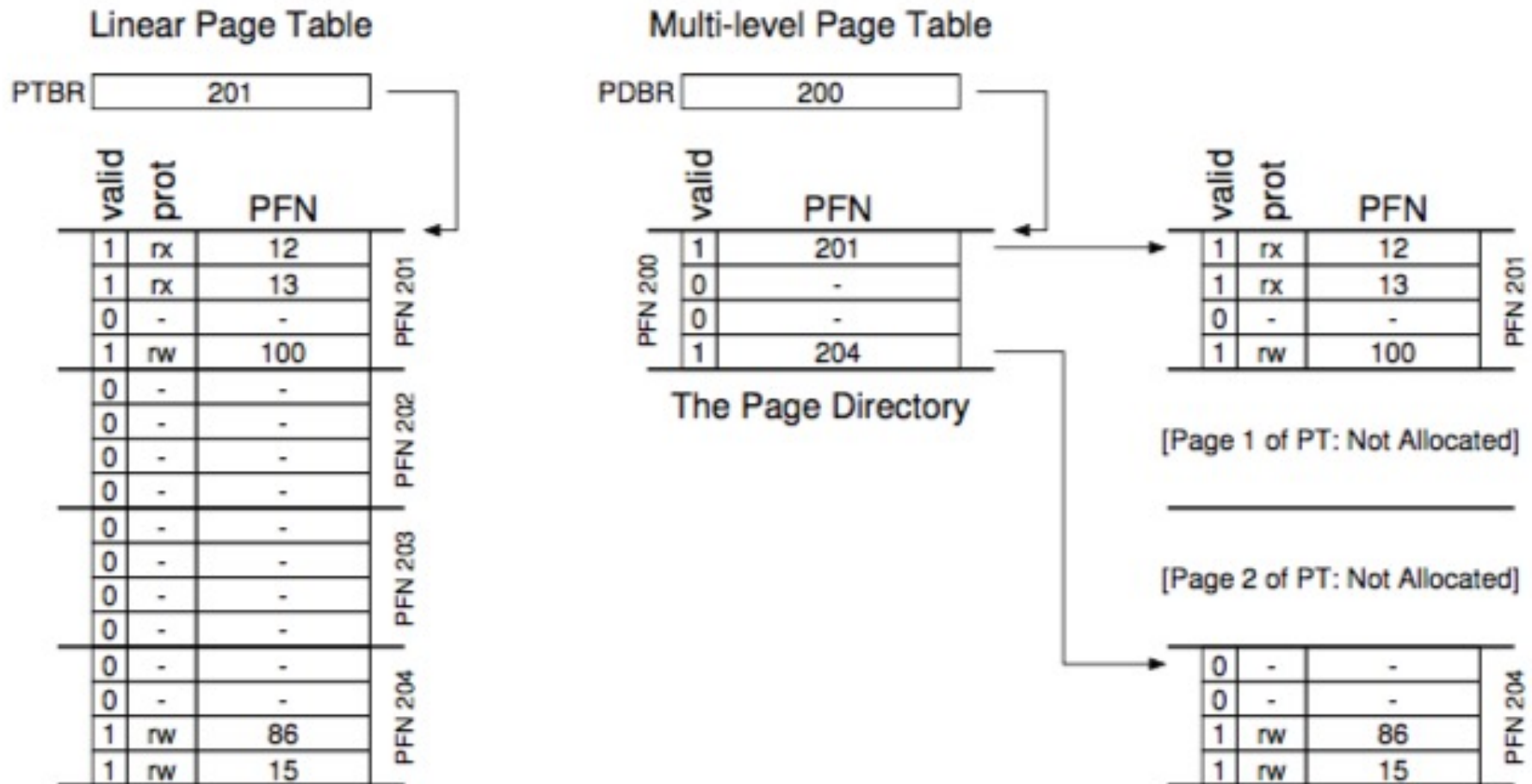
BASE används för att peka ut rätt PT
BOUNDS hur många poster den har!

SN = (VirtualAddress & SEG_MASK) >> SN_SHIFT
VPN = (VirtualAddress & VPN_MASK) >> VPN_SHIFT
AddressOfPTE = Base[SN] + (VPN * sizeof(PTE))



Besparingen kopplat till PT storlek blir ännu bättre med större adressrymder men PT får nu variabel storlek och problem med "luftiga" seg!

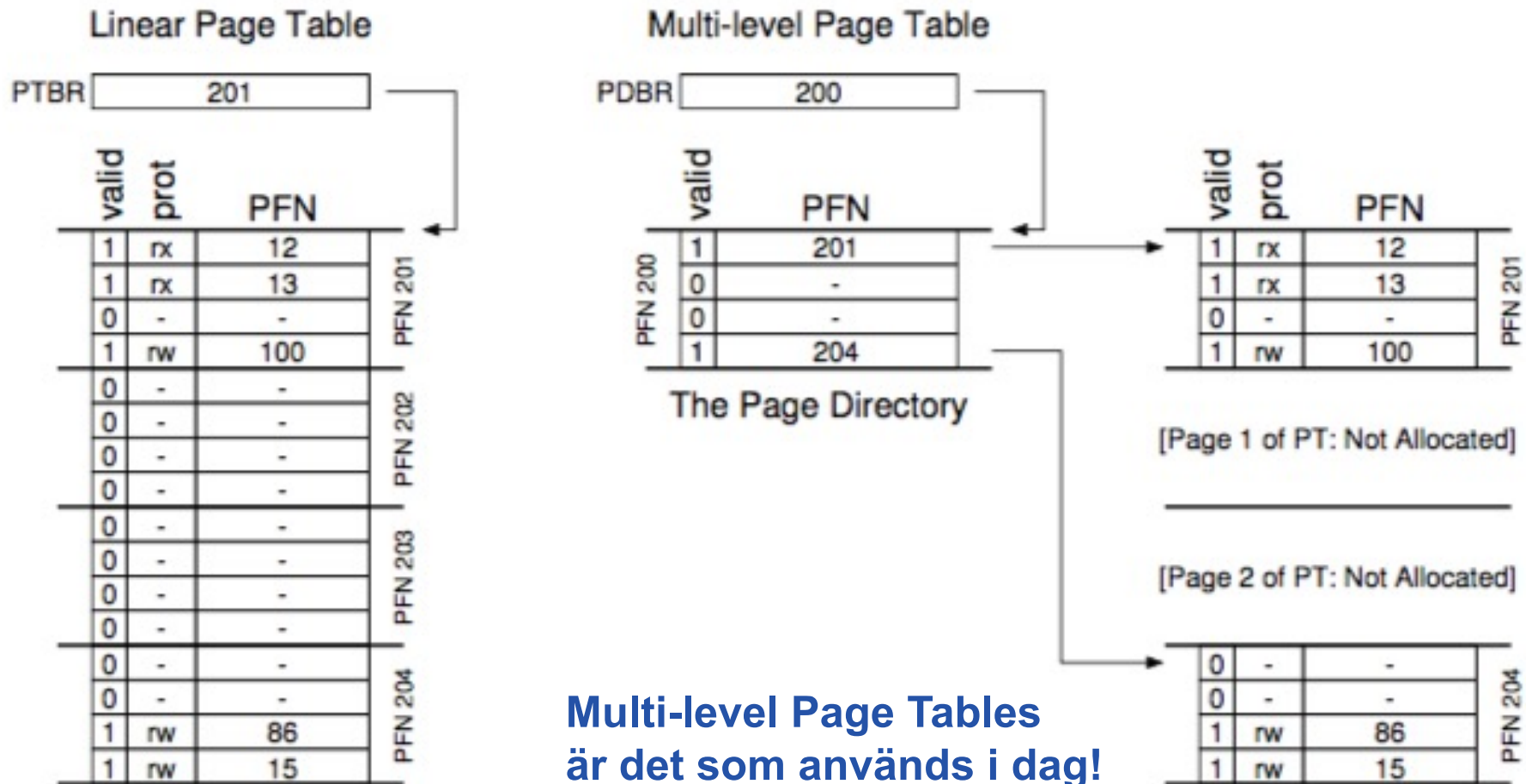
Page Table, trick #2: Multi-level!



Den linjära PT delas upp i PFN-stora delar. Om en hel PFN-stor del av PT är invalid behövs den inte! Ett nytt index behövs för att adm delarna.



Page Table, trick #2: Multi-level!



En "hit" minnesaccess går lika snabbt, en "miss" kräver nu två läsningar i minnet, dock en liten eftergift för minnesbesp. (time-space trade-off).



Övning...

16kB=14bit

VPN=8bit, offset=6

00 0000
11 1111

0000 0000	code
0000 0001	code
0000 0010	(free)
0000 0011	(free)
0000 0100	heap
0000 0101	heap
0000 0110	(free)
0000 0111	(free)

.....

... all free ...

1111 1100	(free)
1111 1101	(free)
1111 1110	stack
1111 1111	stack

Hur många poster skulle en linjärt PT ha?

Hur många skulle vara valid?

Hur stor är en sida?



Övning...

16kB=14bit

VPN=8bit, offset=6

0000 0000	00 0000	code
0000 0001	11 1111	code
0000 0010		(free)
0000 0011		(free)
0000 0100		heap
0000 0101		heap
0000 0110		(free)
0000 0111		(free)
.....		... all free ...
1111 1100		(free)
1111 1101		(free)
1111 1110		stack
1111 1111		stack

Hur många poster skulle en linjärt PT ha?

256 st

Hur många skulle vara valid?

6 st

Hur stor är en sida?

64 Bytes

Om ett PTE är 4 bytes, hur många PTE ryms i en sida?
Hur många sidor behövs för att rymma hela PT?
Vilka av dessa kommer att innehålla en valid mappning?



Övning...

16kB=14bit

VPN=8bit, offset=6

00 0000
11 1111

0000 0000

0000 0001

0000 0010

0000 0011

0000 0100

0000 0101

0000 0110

0000 0111

.....

1111 1100

1111 1101

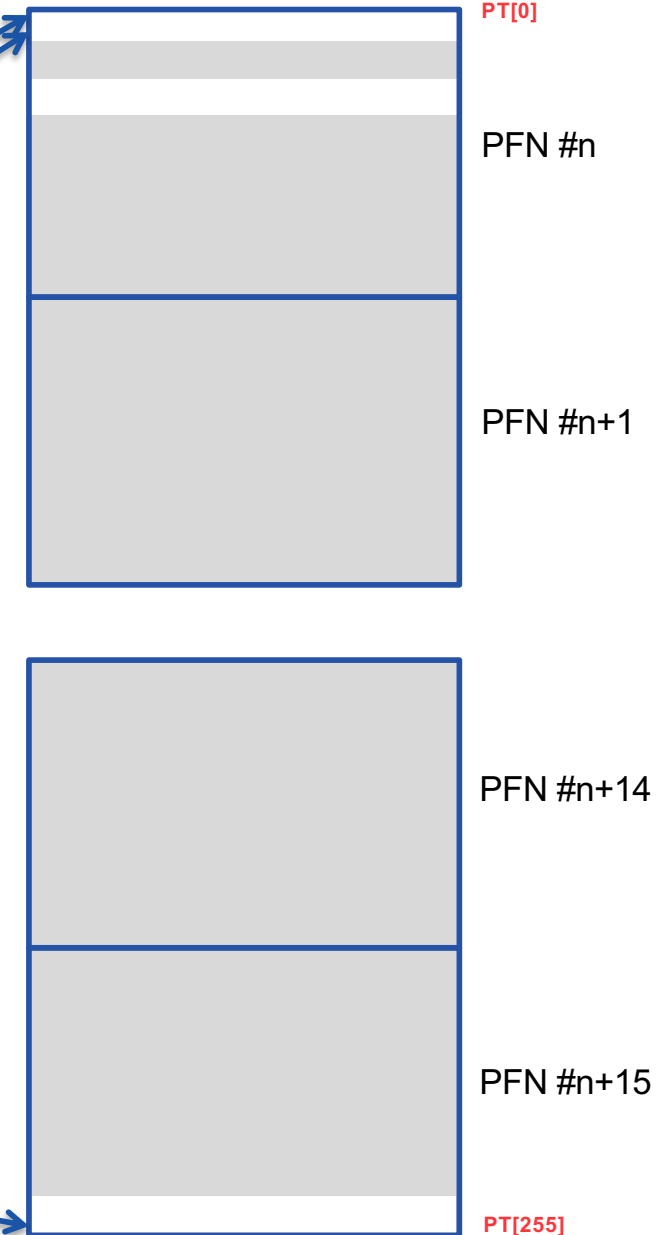
1111 1110

1111 1111

code
code
(free)
(free)
heap
heap
(free)
(free)

... all free ...

(free)
(free)
stack
stack



Om ett PTE är 4 bytes, hur många PTE ryms i en sida?
Hur många sidor behövs för att rymma hela PT?
Vilka av dessa kommer att innehålla en valid mappning?

16 st
16 st
0 & 15



Övning...

16kB=14bit

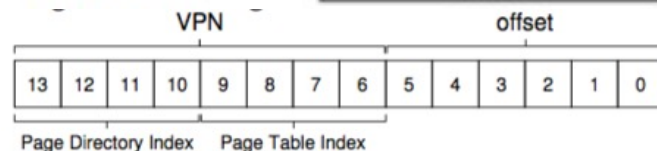
VPN=8bit, offset=6

00 0000
11 1111

0000 0000	code
0000 0001	code
0000 0010	(free)
0000 0011	(free)
0000 0100	heap
0000 0101	heap
0000 0110	(free)
0000 0111	(free)

... all free ...

1111 1100	(free)
1111 1101	(free)
1111 1110	stack
1111 1111	stack



The Page Directory

PFN #n+1

PFN #n

PFN #n+2

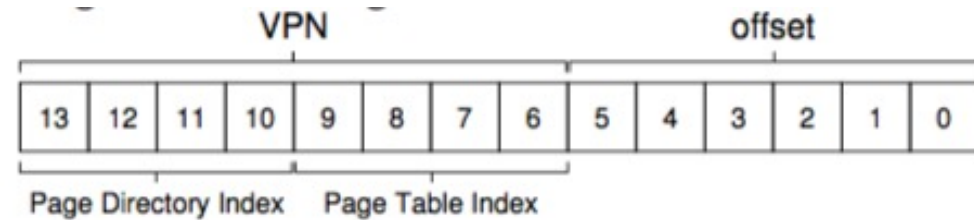
Om vi skulle göra en multi-level PT...

...hur många PDE (4byte) skulle det finnas? 16 st

...hur många PFN skulle det behövas? 1+2



Övning...



16kB=14bit

VPN=8bit, offset=6

00 0000
11 1111

0000 0000	code
0000 0001	code
0000 0010	(free)
0000 0011	(free)
0000 0100	heap
0000 0101	heap
0000 0110	(free)
0000 0111	(free)

... all free ...

1111 1100	(free)
1111 1101	(free)
1111 1110	stack
1111 1111	stack

Page Directory		Page of PT (@PFN:100)			Page of PT (@PFN:101)		
PFN	valid?	PFN	valid	prot	PFN	valid	prot
100	1	10	1	r-x	—	0	—
—	0	23	1	r-x	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	80	1	rw-	—	0	—
—	0	59	1	rw-	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	55	1	rw-
101	1	—	0	—	45	1	rw-

Processen behöver nu komma åt adressen 0x3F80, vart finns den?



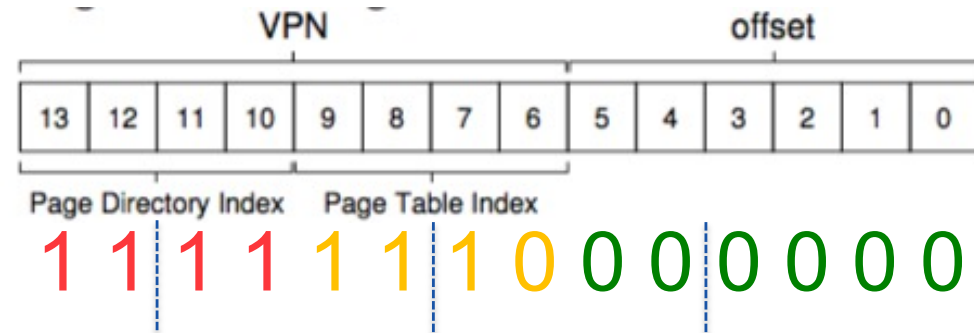
Övning...

16kB=14bit

VPN=8bit, offset=6

00 0000
11 1111

0000 0000	code
0000 0001	code
0000 0010	(free)
0000 0011	(free)
0000 0100	heap
0000 0101	heap
0000 0110	(free)
0000 0111	(free)
... all free ...	
1111 1100	(free)
1111 1101	(free)
1111 1110	stack
1111 1111	stack



Page Directory		Page of PT (@PFN:100)			Page of PT (@PFN:101)		
PFN	valid?	PFN	valid	prot	PFN	valid	prot
100	1	10	1	r-x	—	0	—
—	0	23	1	r-x	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	80	1	rw-	—	0	—
—	0	59	1	rw-	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	55	1	rw-
101	1	—	0	—	45	1	rw-

0 1 0 1 0 1 0 1 0 0 0 0 0 0

Processen behöver nu komma åt adressen 0x3F80, vart finns den?

11 1111 1000 0000 -> Directory entry #15 -> PFN entry #14 -> 55!



Page Table++

Om directory inte "ryms" i en FPN...

...så kan det behövas flera directory nivåer...

...men kostanden för en "miss" blir då större!

Alternativt: Varför har vi inte enbart en inverterad tabell...

...ett entry för varje FPN (som alla proc delar på),
vilken process och VPN som är mappad dit...

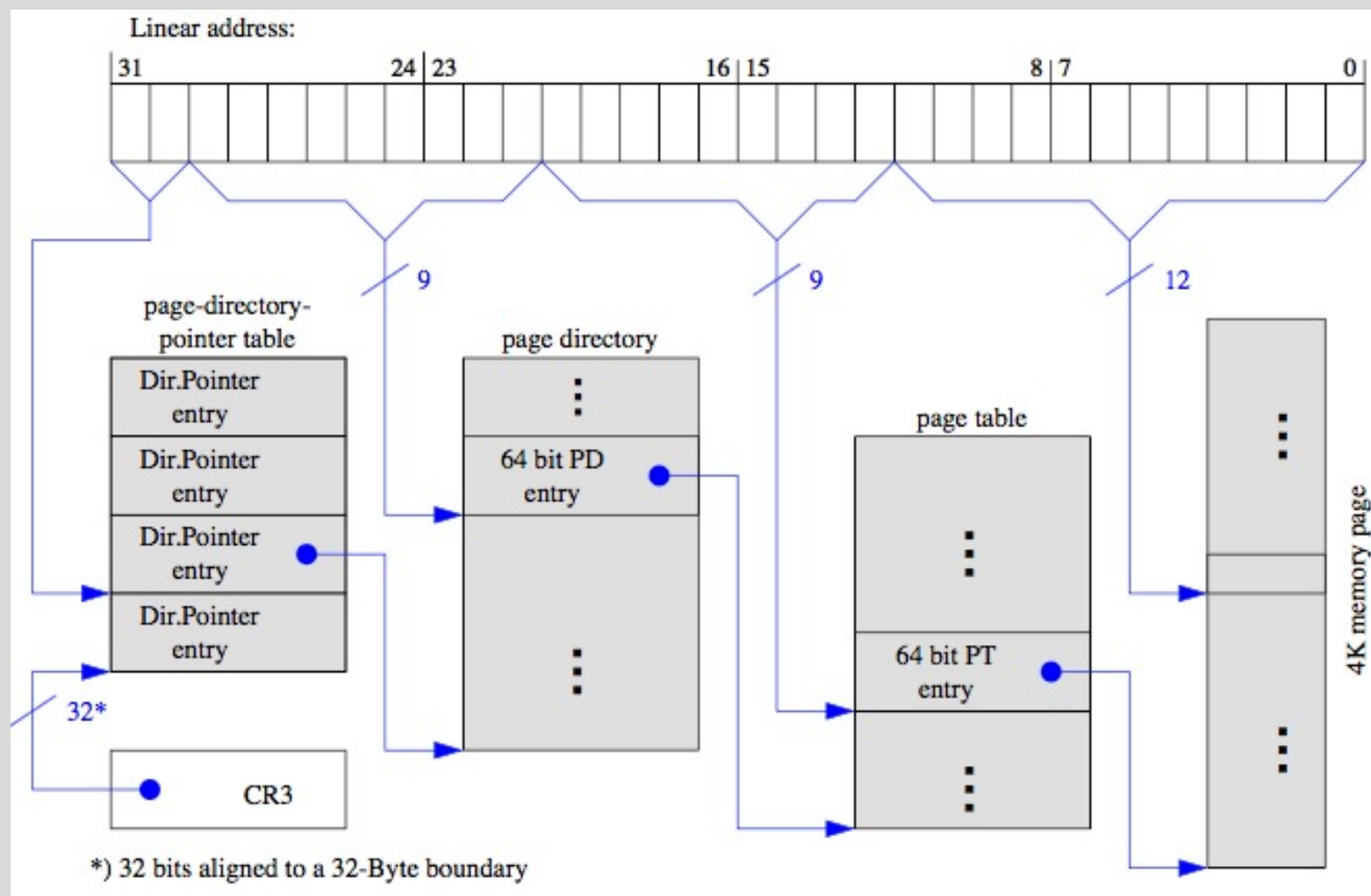
...problemet blir att hitta ett VPN i listan...

...så andra datastrukturer används...

...hash-tabeller (a&d)!

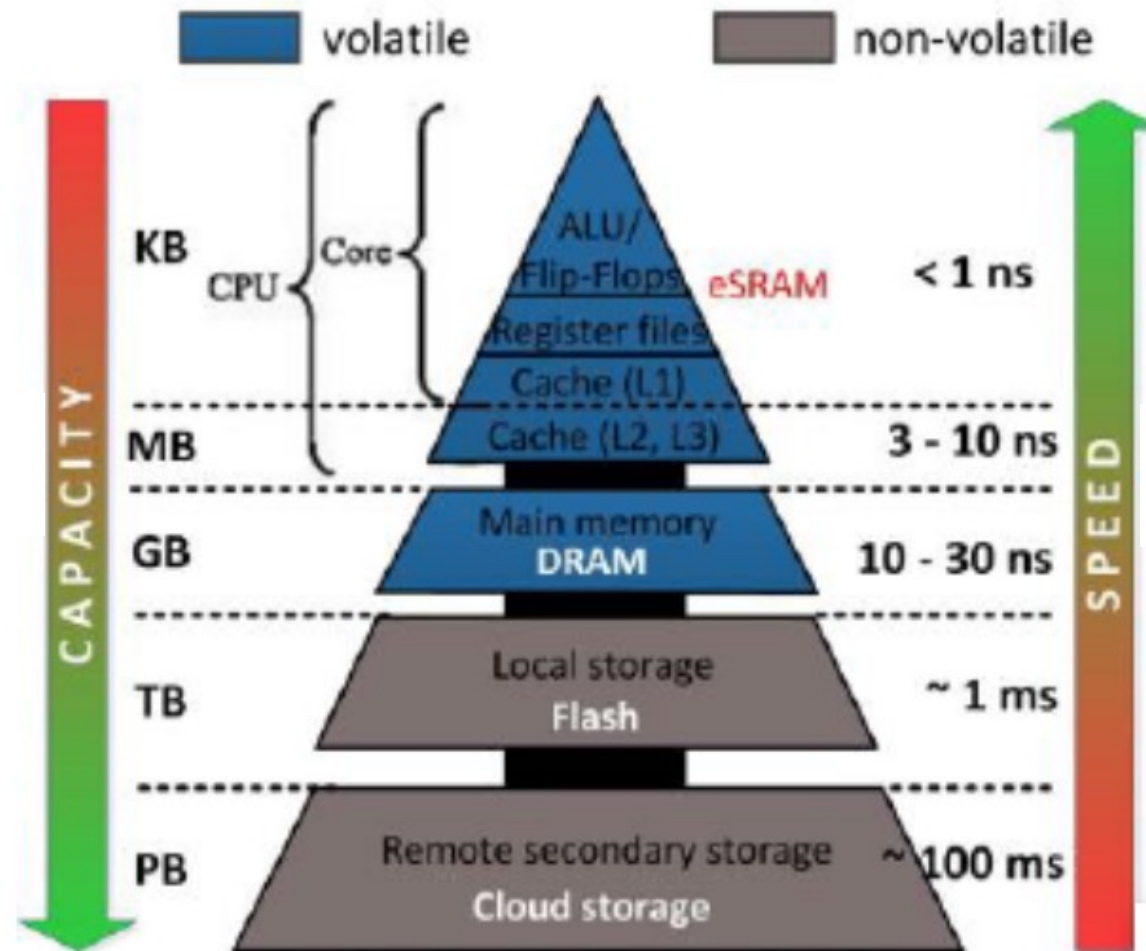
Igen exempel på att vi kan byta utrymme mot tid och vise versa.

x86



När det fysiska minnet tar slut...

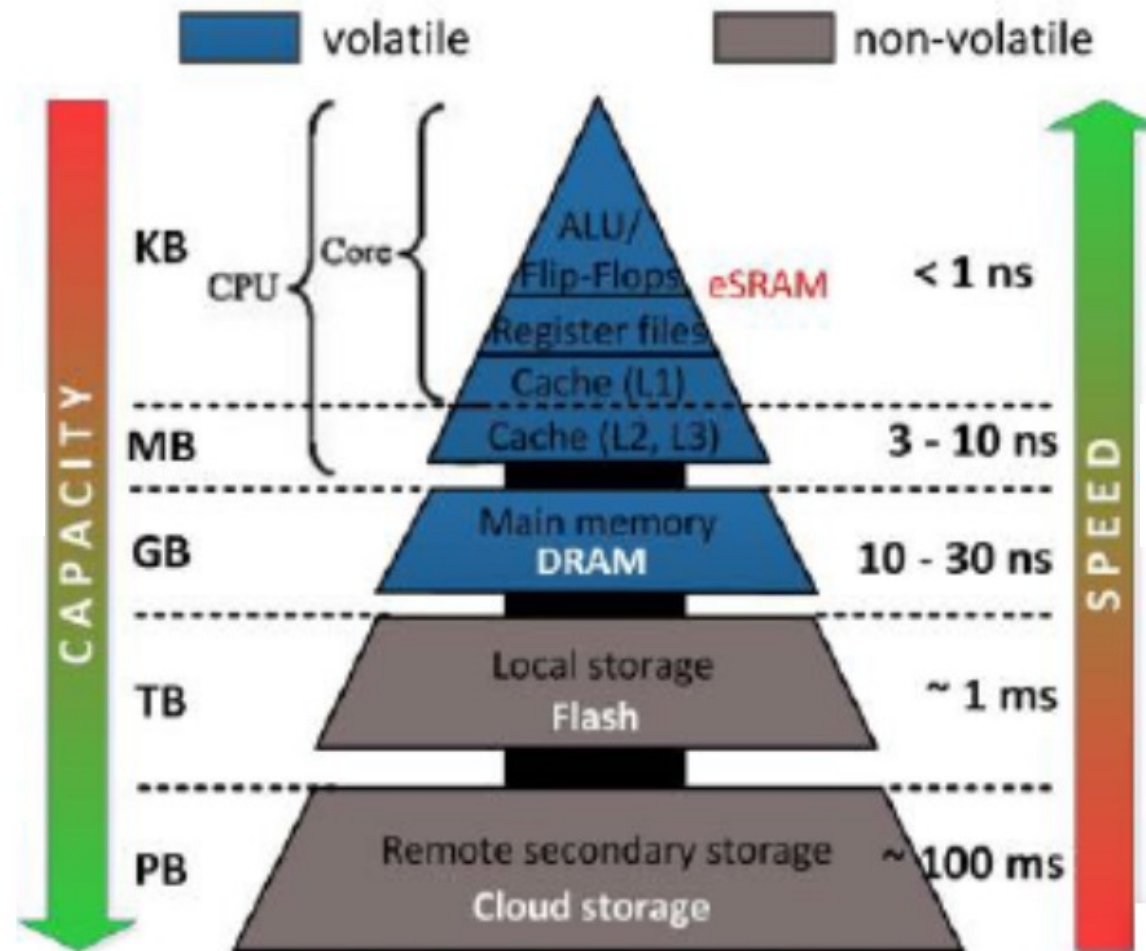
Lagringshierarki



Vanligtvis rymmer inte alla processors valid VP i tillgängligt fysiskt minne...
...sidor som inte behövs precis för stunden kan då lagras på disk!

När det fysiska minnet tar slut...

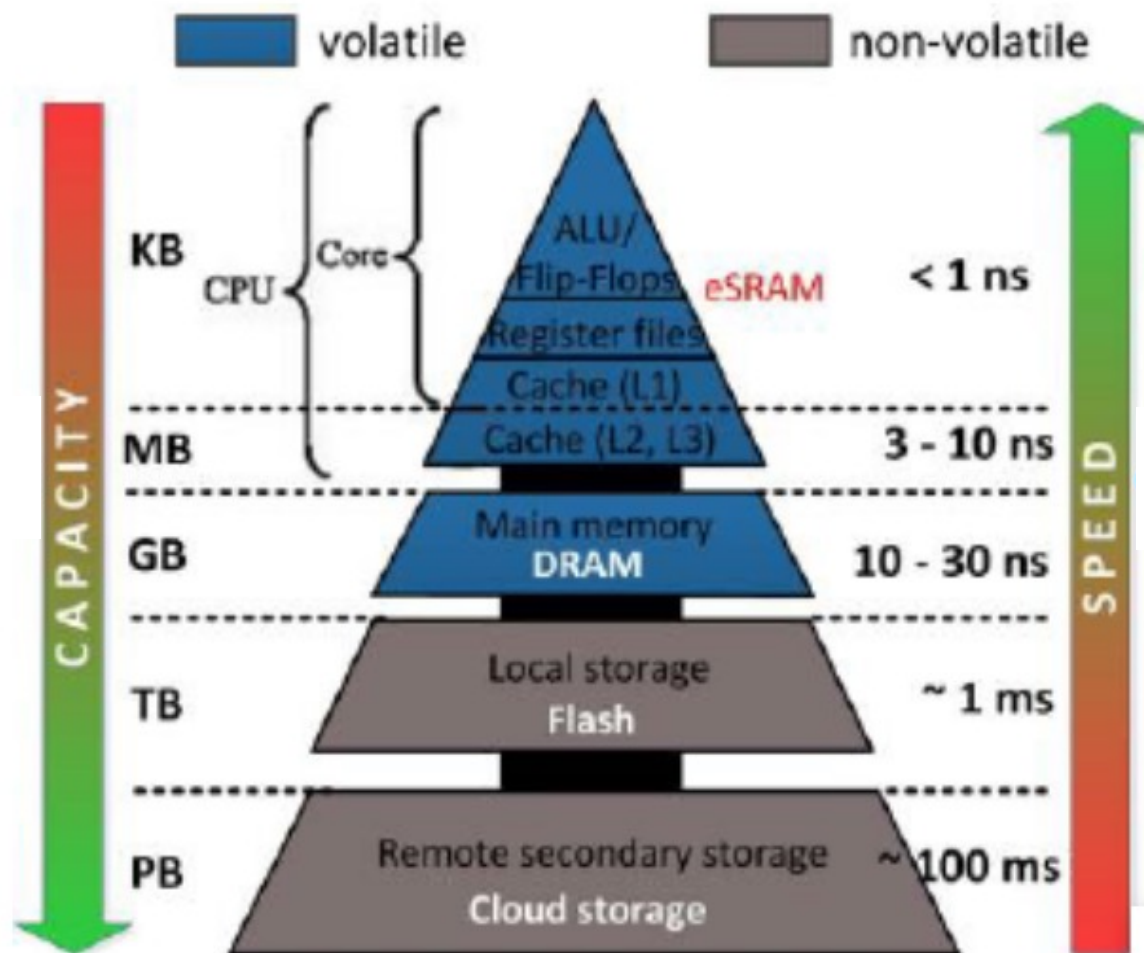
Lagringshierarki



Skär en access till en minnescell som finns i en sida som inte är laddad...
...uppstår ett så kallat Page Fault som leder till att sidan laddas!

När det fysiska minnet tar slut...

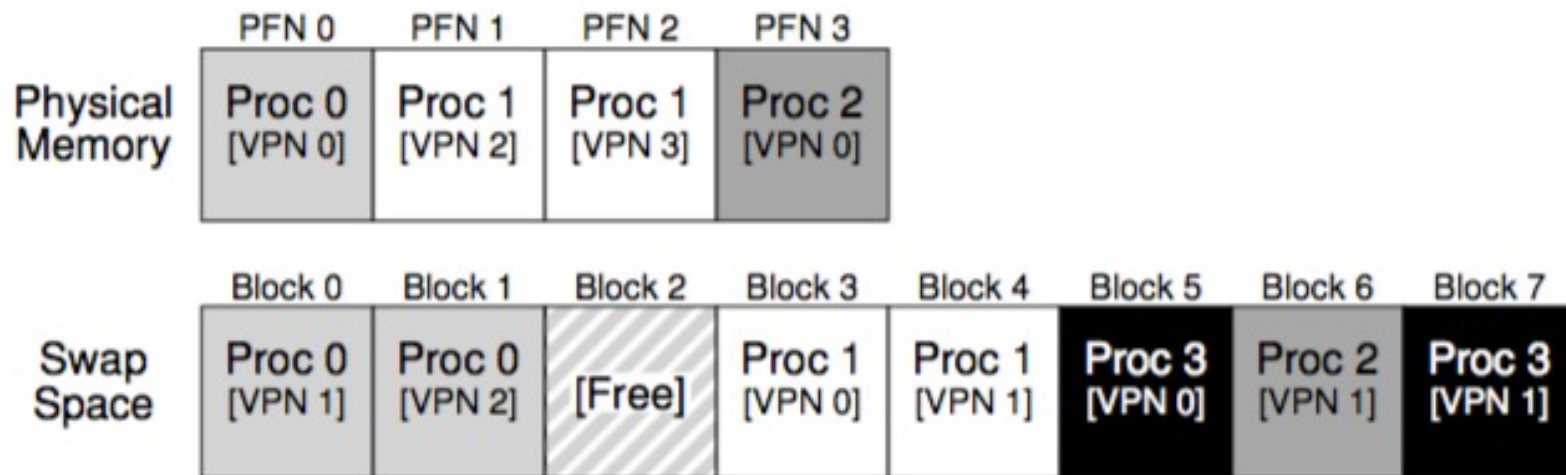
Lagringshierarki



Vilket inte är helt trivialt: Vart finns sidan? Måste någon annan sida slängas ut? Behöver den då sparas? Det kommer att ta LÅNG tid!



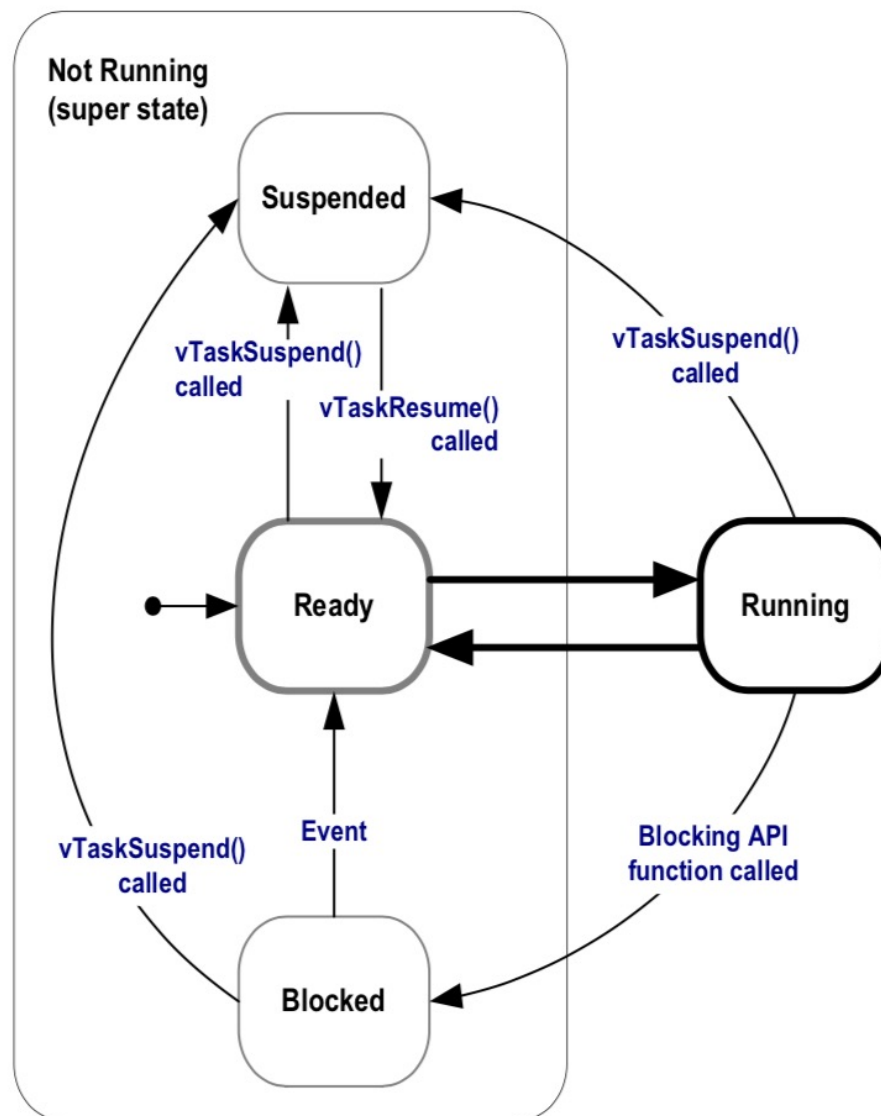
Swap space



Swap Space är en fil på datorns hårddisk organiserad i block som är precis PF-stora och som OS:et lätt kan läsa eller skriva. Det är inte fullt så enkelt som det låter och vi återkommer till det under sista ¼ av kursen. För stunden utgår vi från att denna fil kan bli hur stor som helst!

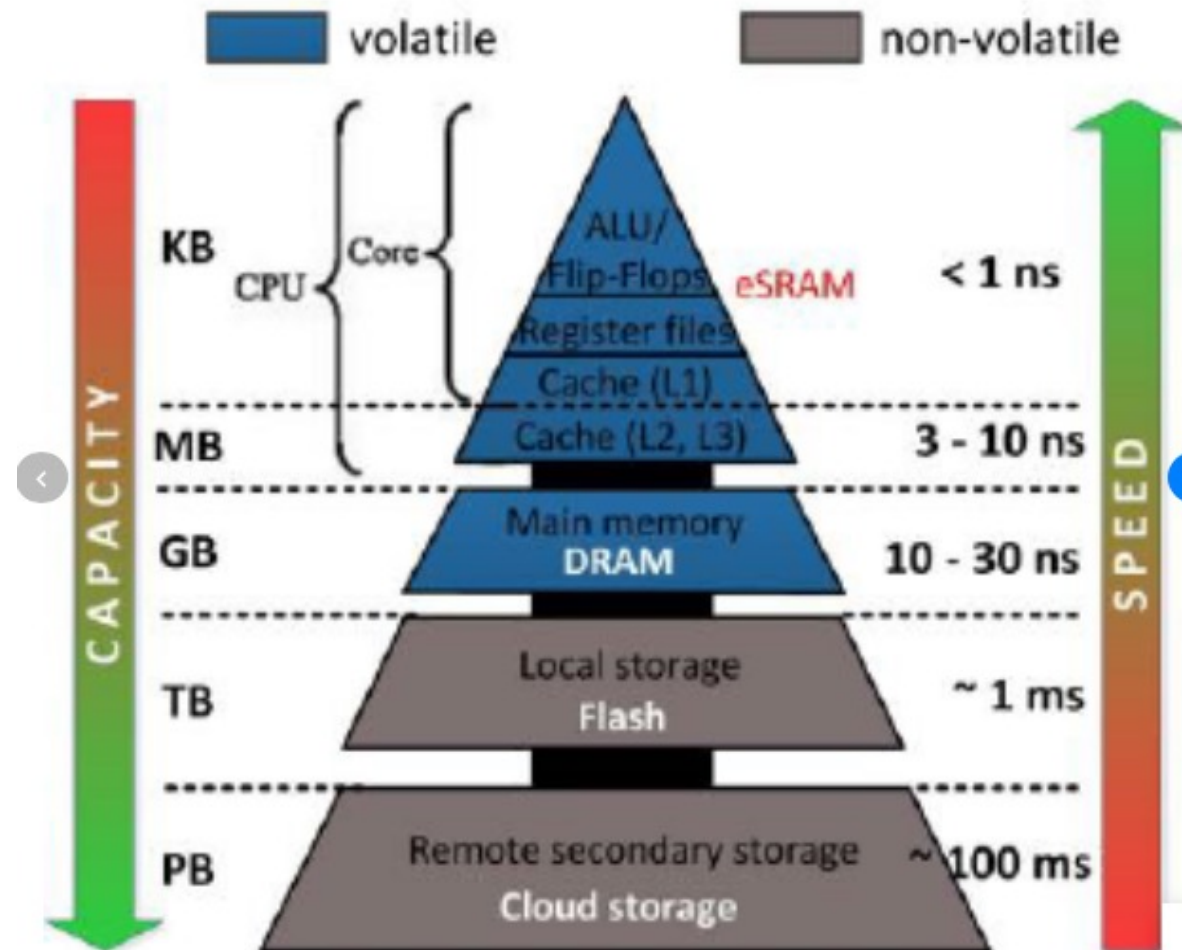
PTE:s P(resens)-bit avslöjar om eftersökt sida finns i minnet eller ej...
...om den inte gör det måste sidan laddas (be swapped in)!

Page Fault



Om $P=0$ kan OS:et använda resten av PTE för att tala om vart den eftersökta sidan finns lagrad. Att hämta sidan är ett jobb för OS:et. Upptäcker TLB en "miss" med $P=0$ genereras ett Page Fault avbrott! Att läsa en sida från disken är komplicerat och tar tid så både av komplexitetsskäl och effektivitetsskäl sker det alltid i mjukvara. Under tiden det tar att läsa in sidan m.m. är berörd process blocked!

Page-replacement policy



Om en sida måste slängas ut, måste den väljas klokt, i annat fall kanske processen börjar köra i "diskfart" i stället för "minnesfart" (100 000ggr)



Sammantaget...

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)    // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset    = VirtualAddress & OFFSET_MASK
6          PhysAddr  = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)           ≈1ns
8      else
9          RaiseException(PROTECTION_FAULT)
10 else                // TLB Miss
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else
16         if (CanAccess(PTE.ProtectBits) == False)
17             RaiseException(PROTECTION_FAULT)
18         else if (PTE.Present == True)
19             // assuming hardware-managed TLB
20             TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
21             RetryInstruction()                       ≈100ns
22         else if (PTE.Present == False)
23             RaiseException(PAGE_FAULT)

1  PFN = FindFreePhysicalPage()
2  if (PFN == -1)                // no free page found
3      PFN = EvictPage()         // run replacement algorithm
4  DiskRead(PTE.DiskAddr, PFN)  // sleep (waiting for I/O)
5  PTE.present = True           // update page table with present
6  PTE.PFN     = PFN            // bit and translation (PFN)
7  RetryInstruction()           // retry instruction           ≈10ms
```

HW

SW

BLOCKED

EvictPage kan ta mycket lång tid så vanligtvis finns en bakgrundsprocess (page daemon) som preventivt slänga ut sidor (styrd av förinställda nivåer, watermark) för att FindFreePP alltid ska kunna retur. en fri sida!



Replacement policy

Vad skulle du lägga 3000Kr på...

- 1) 10% Snabbare processor?
- 2) Byta din mekaniska hårddisk mot en SSD?
- 3) Köpa mer DRAM?



Replacement policy

Vad skulle du lägga 3000Kr på...

1) 10% Snabbare processor?

Datorn blir 1.1ggr snabbare!

1) Byta din mekaniska hårddisk mot en SSD?

2) Köpa mer DRAM?



Replacement policy

Vad skulle du lägga 3000Kr på...

1) 10% Snabbare processor?

Datorn blir 1.1ggr snabbare!

1) Byta din mekaniska hårddisk mot en SSD?

Datorn blir 200ggr snabbare!

2) Köpa mer DRAM?

Average memory access time (AMAT) = $T_m + (P_{miss} * T_d)$; Hit = 99%
 $100ns + (0.01 * 10ms) = 100.1\mu s$ vs $100ns + (0.01 * 40\mu s) = 500ns$



Replacement policy

Vad skulle du lägga 3000Kr på...

1) 10% Snabbare processor?

Datorn blir 1.1ggr snabbare!

1) Byta din mekaniska hårddisk mot en SSD?

Datorn blir 200ggr snabbare!

2) Köpa mer DRAM?

Datorn blir 100ggr snabbare!

Average memory access time (AMAT) = $T_m + (P_{\text{miss}} * T_d)$; Hit = 99.99%
 $100\text{ns} + (0.01 * 10\text{ms}) = 100.1\mu\text{s}$ vs $100\text{ns} + (0.0001 * 10\text{ms}) = 1.1\mu\text{s}$



Uppenbart är en miss otroligt kostsam...

...så frågan är egentligen inte vilken sida vi ska ta in...

...utan vilken sida vi ska kasta ut!

(För det ska helst vara den sidan som inte behövs på länge!)

Vi behöver en policy för vilken sida vi ska kasta ut. För att testa vår policy kan vi hitta på en sekvens av sidaccesser som vi alltid använder. Om vi vet sekvensen kan vi också skapa en optimal policy som vi kan jämföra mot...



(Omöjlig att implementera.)

Optimal policy (for 01201303121 / 3 page cache)

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2
1	Hit		0, 1, 2
3	Miss	2*	0, 1, 3
0	Hit		0, 1, 3
3	Hit		0, 1, 3
1	Hit		0, 1, 3
2	Miss	3	0, 1, 2
1	Hit		0, 1, 2

} Cold-start miss aka Compulsory miss

We can also calculate the hit rate for the cache: with 6 hits and 5 misses, the hit rate is $\frac{Hits}{Hits+Misses}$ which is $\frac{6}{6+5}$ or 54.5%. You can also compute the hit rate *modulo* compulsory misses (i.e., ignore the *first* miss to a given page), resulting in a 85.7% hit rate.

* Vi behöver både 0 & 1 innan vi behöver 2 igen, så släng ut 2!



(Lätt att implementera, användes i de första datorerna)

FIFO policy?! (for 01201303121 / 3 page cache)

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		First-in→	0
1	Miss		First-in→	0, 1
2	Miss		First-in→	0, 1, 2
0	Hit		First-in→	0, 1, 2
1	Hit		First-in→	0, 1, 2
3	Miss	0	First-in→	1, 2, 3
0	Miss	1	First-in→	2, 3, 0
3	Hit		First-in→	2, 3, 0
1	Miss	2	First-in→	3, 0, 1
2	Miss	3	First-in→	0, 1, 2
1	Hit		First-in→	0, 1, 2

} Cold-start miss aka Compulsory miss

Comparing FIFO to optimal, FIFO does notably worse: a 36.4% hit rate (or 57.1% excluding compulsory misses). FIFO simply can't determine the importance of blocks: even though page 0 had been accessed a number of times, FIFO still kicks it out, simply because it was the first one brought into memory.

Det är egentligen ännu sämre, det finns sekvenser av sidreferenser, där det i princip alltid blir fel. Problemet benämns Belady's anomali.

Random policy?! (for 01201303121 / 3 page cache)

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2
1	Hit		0, 1, 2
3	Miss	0	1, 2, 3
0	Miss	1	2, 3, 0
3	Hit		2, 3, 0
1	Miss	3	2, 0, 1
2	Hit		2, 0, 1
1	Hit		2, 0, 1

Cold-start miss aka
Compulsory miss

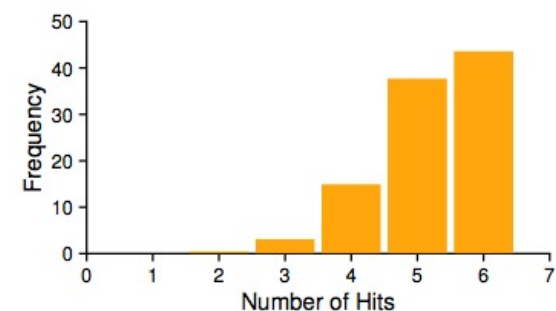


Figure 22.4: Random Performance Over 10,000 Trials

I snitt bättre än FIFO och lite sämre än optimal,
men kan om oturen är framme vara riktigtusel...



(Möjlig att implementera, även om det kräver en hel del administration.)

LRU(LFU) policy?! (for 01201303121 / 3 page cache)

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		LRU→	0
1	Miss		LRU→	0, 1
2	Miss		LRU→	0, 1, 2
0	Hit		LRU→	1, 2, 0
1	Hit		LRU→	2, 0, 1
3	Miss	2	LRU→	0, 1, 3
0	Hit		LRU→	1, 3, 0
3	Hit		LRU→	1, 0, 3
1	Hit		LRU→	0, 3, 1
2	Miss	0	LRU→	3, 1, 2
1	Hit		LRU→	3, 2, 1

} Cold-start miss aka Compulsory miss

För denna sekvens alltid lika bra som optimal policy...
...vi kan inte se in i framtiden men tids- och platslokalitet finns ofta i kod!

I verkligheten...

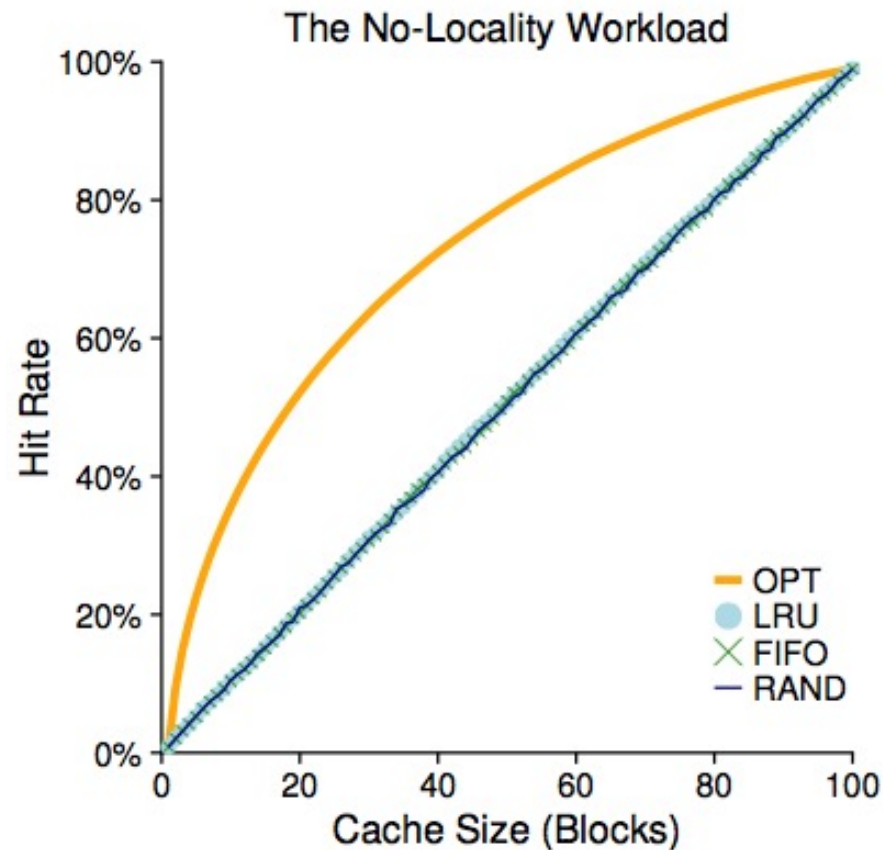


Figure 22.6: The No-Locality Workload

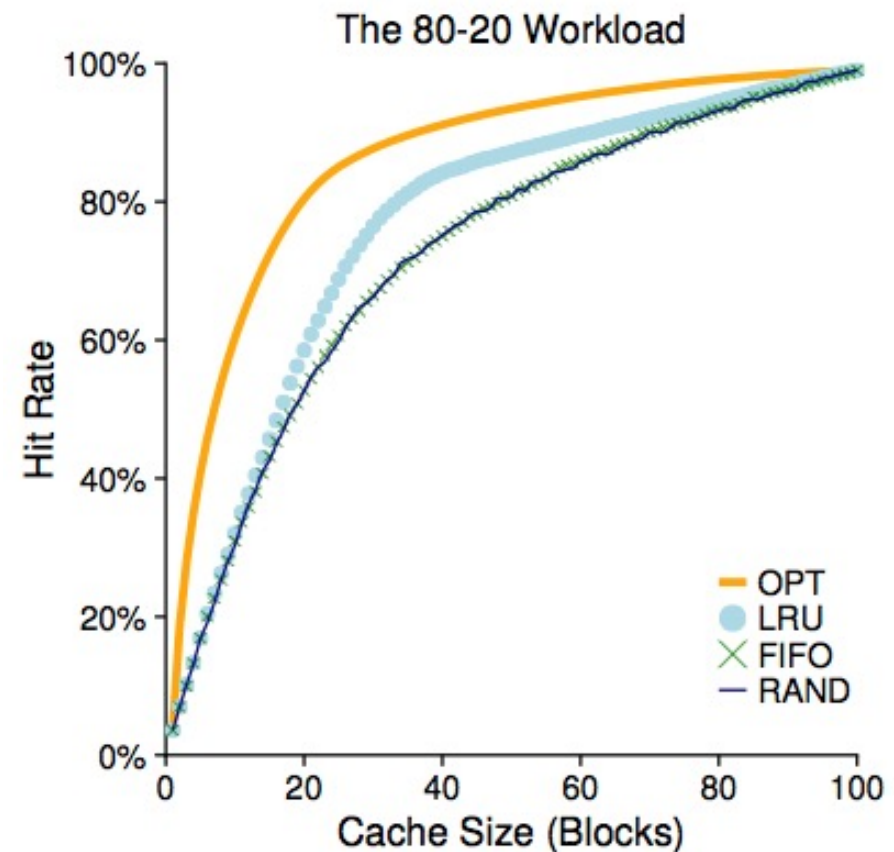


Figure 22.7: The 80-20 Workload

Så fort det finns någon form av lokalitet går det att bli bättre än slumpen!
Kom ihåg att en förbättring på 1/10-dels procent kan göra underverk!!!

Approximating LRU!

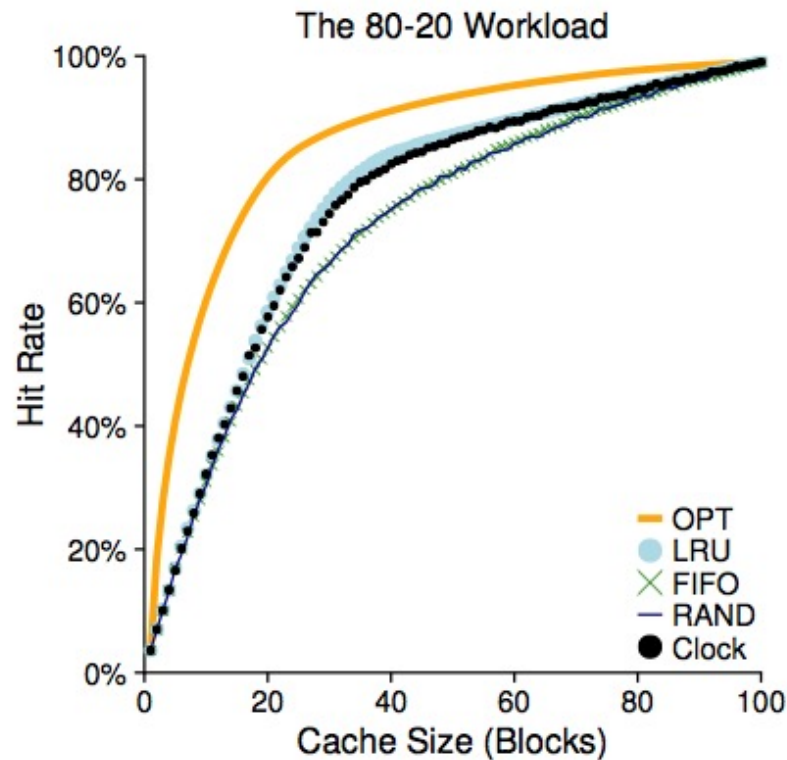


Figure 22.9: The 80-20 Workload With Clock

Vi vill ha en LRU policy men det är svårt i praktiken att tidsstämpla varje access, för att inte tala om problemet att undersöka alla tidsstämplar när en sida ska slängas ut!

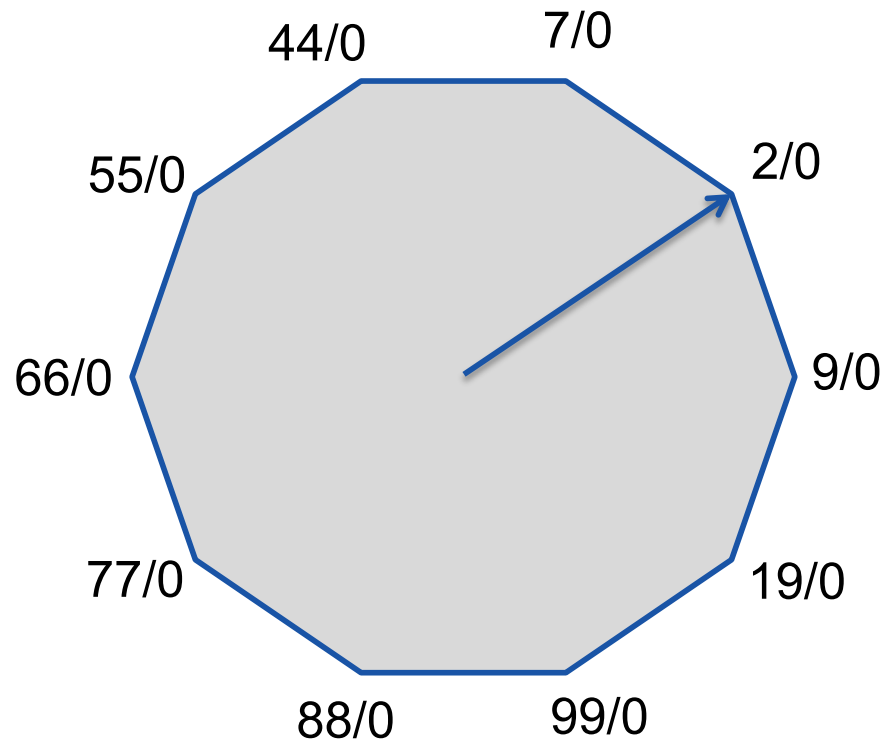
Vi behöver en policy som liknar LRU men är enklare att implementera!

Det finns en sådan app. LRU policy som ofta kallas klockalgoritmen...



Klockalgoritmen

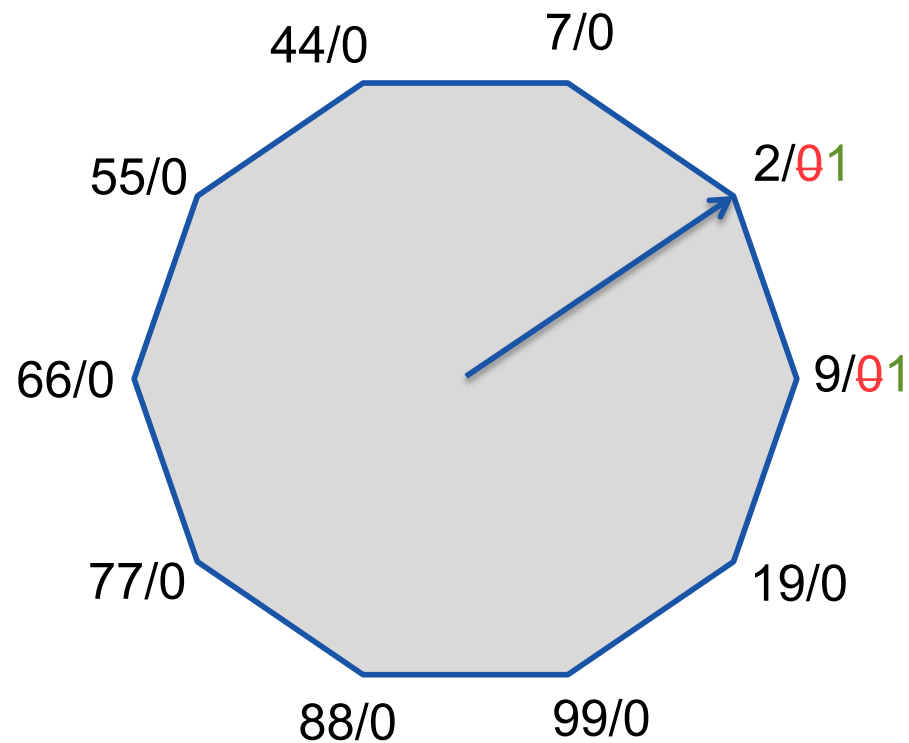
- Tänk dig sidreferenserna ordnade som en cirkulär kö som talen på en urtavla. Varje sida har en referens-bit som initialt sätt till 0...



(Sidorna 7,2,9,19,99,88,77,66,55,44 har nyligen efterfrågats.)



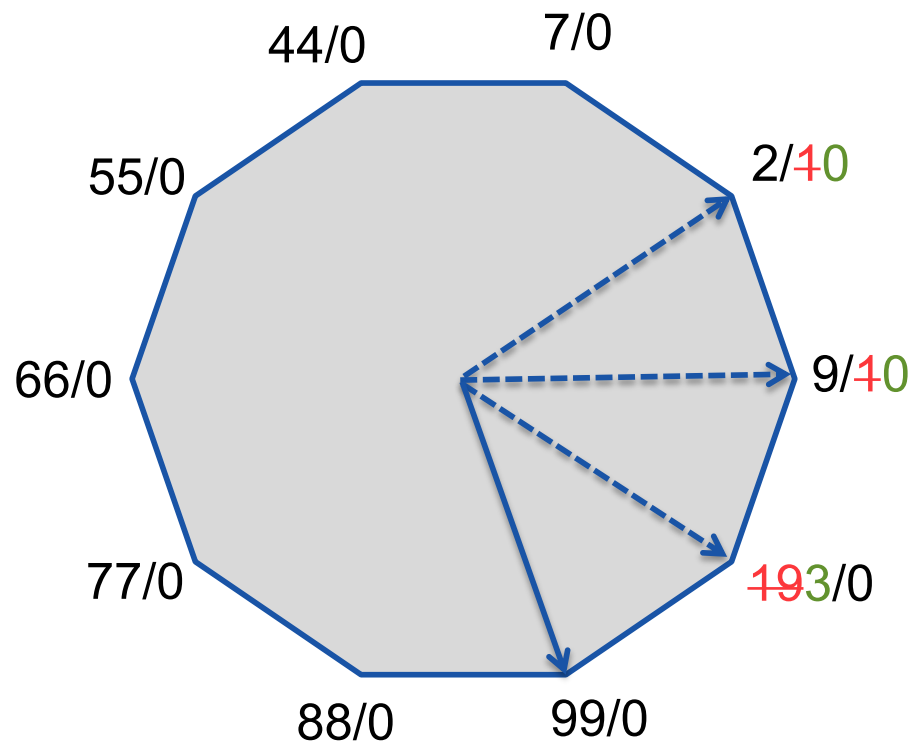
Klockalgoritmen



- Tänk dig sidreferenserna ordnade som en cirkulär kö som talen på en urtavla. Varje sida har en referens-bit som initialt sätt till 0...
- Varje gång en sida refereras sätt referens-biten till 1...
- Read 2, träff, set
- Write 9, träff, set



Klockalgoritmen



- Tänk dig sidreferenserna ordnade som en cirkulär kö som talen på en urtavla. Varje sida har en referens-bit som initialt sätt till 0...
- Varje gång en sida refereras sätt referens-biten till 1...
- När en sida ska slängas ut förs en "visaren" framåt på urtavlan tills den hittar en sida där referensbiten inte är satt, samtidigt som den nollställer referensbiten på de referenser som den passerar!
- Read 2, träff, set
- Write 9, träff, set
- Read 3, miss...
 - 2, RU, nollställ
 - 9, RU, nollställ
 - 19, out, 3 in, ptr 99!

Sidan som "visaren" stannar på slängs ut... ...referenserna "bakom" visaren måste refereras igen innan visaren hinner dit på nytt, annars...



Dirty, Other VM Policies & Obstacles

- Avoid dirty=wr pages if possible (pre-cleaning)
- Page selection
 - Demand paging (lazy evaluation)
 - Prefetching (anticipatory)
- Thrashing (extensive page faults)
 - Admission control
 - Brute force



Begrepp

Paging model

Linear Page Table, Address translation

Virtual Page vs. Page Frame

Page Table Base Register

Page, Segmentation, Protection fault

Translation Look-aside buffer (TLB)

Spatial & temporal locality

Least recently used vs. Random

Multi-level Page Table

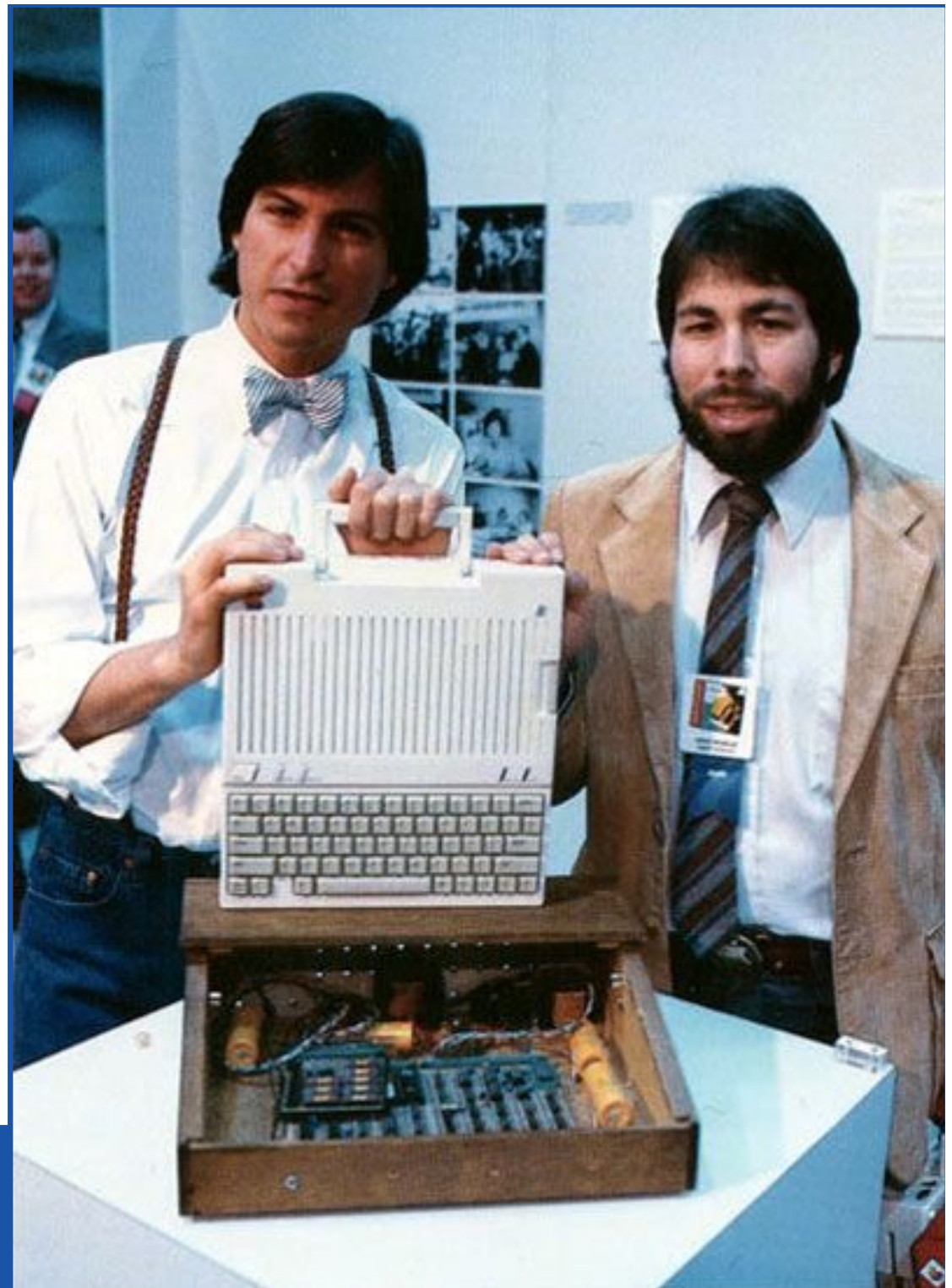
Storage Hierarchies

Swap space

Page replacement policies

FIFO vs. Random vs. LRU/LFU

The Clock Algorithm





Historik

1810xx 0.1 AC Outline

1812xx 1.0 AC Release 1.0